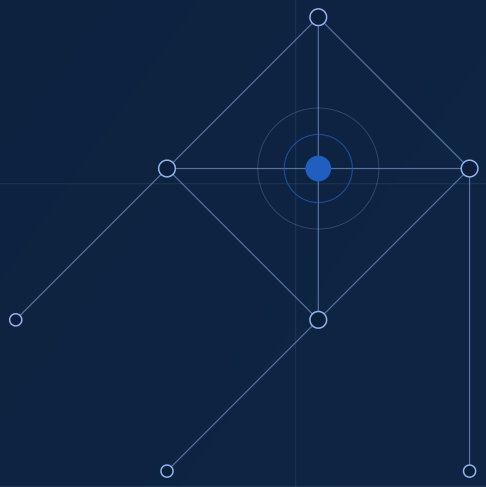
DOCUMENTO DE CONTEXTO PARA CLAUDE



GUÍA PRÁCTICA · 9 PARTES

Vídeo con IA y automatización con Power Automate

Herramientas gratuitas, flujos paso a paso, agentes y casos de portfolio, reunidos para subir a un proyecto de Claude y trabajar con ellos.

EDICIÓN
Octubre 2026

CONTENIDO
HyperFrames · Power Automate · n8n · Three.js

USO
Subir a un proyecto o chat de Claude

00 Acerca de este documento

Documento de contexto para subir a Claude (proyecto o chat) y construir un portfolio. Reúne dos temas principales: **crear vídeos gratis** con HyperFrames y herramientas libres, y **construir, editar y depurar flujos** con el servidor MCP de Power Automate. Las partes siguientes añaden casos reales, automatizaciones propias y recetas listas para copiar.

ANTES DE AFIRMAR CIFRAS

Las cifras, precios, licencias y versiones cambian a menudo. Verifícalos en la fuente oficial antes de usarlos. Fuentes principales: PDF de ghostless sobre HyperFrames, guía de El Telar (precios a 17-oct-2026) y un vídeo tutorial sobre el plugin de Power Automate.

Contenido

00 **Antes de empezar** — 03
Anatomía de un agente de IA y glosario rápido

01 **Vídeo con IA gratis** — 05
HyperFrames, modelos de pago, herramientas libres y etiquetado en la UE

02 **Power Automate con el servidor MCP** — 09
Instalación, entornos, cinco demos y cómo llevarlo al portfolio

03 **Caso: La Fábrica de agentes** — 13
Telegram, subagentes y panel visual, en ocho pasos reproducibles

04 **Cómo hacer mis automatizaciones** — 21
n8n con Docker, ciclo de creación y errores frecuentes

05 **Prompts de innovación y marketing** — 25
Proceso IDEO, tendencias, problem framing y diseño de producto

06 **Caso: FitBattle Gym** — 28
Modelo entidad-relación, 3FN y automatizaciones asociadas

07 **Caso: Dashboard financiero de Inditex** — 31
Power BI, modelo DAX y entorno de trabajo con MCP

08 **Cómo hacer los vídeos 3D** — 37
Three.js a MP4 vertical, con bucle perfecto

09 **Guía de las herramientas de El Telar** — 41
Catorce herramientas, flujo completo, licencias y glosario

Cómo leer esta guía

INFO Contexto útil o definición.

AVISO Límite, coste o riesgo.

CONSEJO Atajo o buena práctica.

ERROR Síntoma frecuente y arreglo.

NOTA Aclaración secundaria.

Los pasos aparecen como secuencias numeradas. Los comandos y el código están en bloques oscuros y los textos para copiar en cajas de «Prompt». Sustituye siempre lo que va entre [corchetes].

ANTES DE EMPEZAR

Anatomía de un agente de IA

00

Cinco piezas explican casi todo lo que aparece en esta guía. Un agente de IA no es un modelo más listo: es un cerebro al que se le suma una biblioteca, un sistema nervioso, memoria muscular y manos.

Esquema inspirado en la infografía «Anatomía de un agente de IA» de AI Mate.

PIEZA	ES COMO…	QUÉ HACE	EN ESTA GUÍA
LLM	El cerebro	Lee, escribe y razona. Aún no sabe nada de tu negocio. Es la base del agente que creas.	Claude (Opus, Sonnet): el modelo que piensa.
RAG	Cerebro + biblioteca	Busca información antes de responder. Trabaja con tus datos, no con suposiciones.	Este PDF subido como contexto, `feedback.md` y `state.json`.
MCP	El sistema nervioso	Estándar para conectar herramientas y datos. Transporta señales entre el cerebro y tus aplicaciones y activa las herramientas conectadas.	Plugin de Telegram, servidor Flow Agent de Power Automate, Supabase.
Skills	Memoria muscular	Conocimiento guardado una vez y reutilizado siempre, sin volver a explicar la tarea.	`fabrica-ideas`, `fabrica-guion`, `fabrica-video`, `hyperframes`.
Agente de IA	Cerebro + manos	Planifica los pasos y luego los ejecuta en tus herramientas.	El capataz y los subagentes de la fábrica (Parte 3).

LLM cerebro + **RAG** biblioteca + **MCP** sistema nervioso + **Skills** memoria muscular = **Agente** cerebro + manos

FÓRMULA PARA RECORDARLO

Agente = LLM + RAG + MCP + Skills. Cuanto más claro tengas qué pieza falta, más fácil es arreglar un agente que no funciona:

- Si no conoce tus datos, falta **RAG**.
- Si no puede actuar en una app, falta **MCP**.
- Si repites siempre la misma explicación, falta una **skill**.

Glosario rápido

TÉRMINO	QUÉ ES
JSON	Formato de texto para guardar y enviar datos como pares clave y valor. Se usa en configuración (`access.json`, `settings.json`) y en las APIs. Reglas: comillas dobles, comas entre elementos y ninguna coma después del último.

TÉRMINO	QUÉ ES
API / endpoint	La puerta por la que un programa habla con otro. Un endpoint es una dirección concreta de esa puerta, por ejemplo `POST /api/event` en el panel.
Token	Contraseña larga que identifica a tu bot o servicio. Va en un archivo `.env`, nunca en chats ni repositorios; si se expone, se revoca y se crea otro.
.env	Archivo de texto con secretos (`CLAVE=valor`) que el programa lee al arrancar.
CLAUDE.md	Instrucciones fijas que Claude Code lee al abrir una carpeta. Es donde se define el capataz.
Subagente	Un Claude auxiliar que recibe una tarea concreta, la hace aparte y devuelve el resultado. Permite trabajar en paralelo.
Modo headless (`claude -p`)	Ejecutar Claude sin conversación: recibe un texto, trabaja y termina. Es lo que usa la caja «Pedir» del panel.
Allowlist	Lista de personas autorizadas (por su ID). Quien no está, no puede hablar con el bot.
localhost	Tu propio ordenador. `http://localhost:3000` es una web que solo ves tú.
Hook	Acción automática que se dispara cuando ocurre algo (por ejemplo, al editar un archivo).

Ejemplo de JSON: un aviso de un agente al panel

JSON

```
{
  "factory": "ideas",      // qué fábrica avisa
  "status": "working",     // working | review | done | error
  "step": 2,               // paso actual, de 1 a 5
  "input": "ahorrar dinero",
  "note": "filtrando ideas"
}
```

> **NOTA**
> El JSON real no admite comentarios; aquí se añaden con `//` solo para explicar.

PARTE 1

Vídeo con IA gratis

01

Cómo crear vídeos con **HyperFrames** y herramientas libres, qué modelos de pago existen cuando se necesita metraje real y qué obliga la ley al publicarlos.

1.1 HyperFrames, el motor gratis

HyperFrames, de HeyGen (licencia Apache 2.0, código abierto), convierte código (HTML, CSS y animaciones) en MP4. Se le describe el vídeo a un asistente de IA, escribe las escenas y HyperFrames renderiza cada fotograma en el propio ordenador. Sin app de edición, sin suscripción y sin créditos. Repositorio: `github.com/heygen-com/hyperframes`.

Requisitos

- **Node.js 22** o superior.
- **FFmpeg** (`winget install ffmpeg` en Windows, `brew install ffmpeg` en Mac).
- Un **asistente de IA para programar**. Claude Code funciona mejor (plugin oficial); Cursor, Codex y otros usan `npx skills add`.

Instalación y primer render

1. **Instala el plugin en Claude Code**

```
TERMINAL

claude plugin marketplace add heygen-com/hyperframes
claude plugin install hyperframes@hyperframes
```

2. **Arranca la skill** escribiendo `/hyperframes:hyperframes`, o usa la alternativa sin plugin:

```
TERMINAL

npx hyperframes init mi-video
cd mi-video
```

3. **Previsualiza y renderiza**

```
TERMINAL

npx hyperframes preview
npx hyperframes render
```

Tres prompts de partida

1 · DOCUMENTAL

Make a 60-second documentary called "[title]" about [topic]. Research the story first, then write a script in 6 scenes. For each scene: a kinetic title, a short narration line, a timeline or map graphic where it helps, and slow camera moves on the footage I give you in /media. Dark cinematic style, one accent colour. Add a voiceover track and soft music, then render.

2 · EXPLAINER DE PRODUCTO

Make a 30-second vertical (1080x1920) explainer for [product]. Hook in the first 2 seconds, then 3 benefits with animated UI mockups, then a call to action: "[CTA]". Clean editorial style, word-by-word captions synced to the voiceover. Render at 30 fps.

3 · REEL SIN CARA

Make a 35-second faceless reel for Instagram about [topic]. Big word-by-word captions, one idea per scene, a counter or stamp animation on every key number, and a dark end card that says "Comment [WORD]". Keep every scene editable so I can ask for changes like "make scene 3 night-time".

Cambios posteriores en lenguaje natural: «escena 12 de noche», «títulos más lentos en la escena 2», «cambia la música por algo más calmado». Después se vuelve a renderizar.

ADVERTENCIAS

- HyperFrames **no genera metraje**: construye gráficos, títulos, timelines y layouts alrededor del material que se aporta (carpeta `/media`). Las tomas del vídeo original se hicieron con Google Flow.
- El render corre en el ordenador (Chrome headless). Es gratis, pero los vídeos largos tardan en portátiles viejos. HeyGen ofrece render en la nube de pago, opcional.
- El asistente de IA puede costar dinero: HyperFrames es gratis, el plan de Claude Code u otro no.

Skills del plugin

Con el plugin se puede invocar directamente cualquiera de estas skills:

SKILL	PARA QUÉ SIRVE
`faceless-explainer`	Explainer sin cara a partir de un texto.
`product-launch-video`	Vídeo promocional desde una URL.
`motion-graphics`	Gráfico animado corto.
`music-to-video`	Vídeo sincronizado a una canción.
`talking-head-recut`	Gráficos sobre un vídeo propio.
`embedded-captions`	Subtítulos incrustados.
`pr-to-video`	Vídeo desde un pull request.
`general-video`	Todo lo demás.

1.2 Modelos de vídeo con IA (de pago, para metraje real)

MODELO	DURACIÓN	FUERTE	DÉBIL
Veo 3.1 (Google)	4, 6 u 8 s; extensible a ~148 s encadenando	Audio nativo, imagen de referencia, último fotograma, SynthID	Muchos personajes interactuando

MODELO	DURACIÓN	FUERTE	DÉBIL
Gemini Omni Flash	Tramos de 3 a 10 s	Edición conversacional, entrada mixta texto, imagen, audio y vídeo	Solo inglés garantizado; en el EEE no edita vídeos subidos
Kling 3.0	3 a 15 s	Movimiento, varios personajes	Seguir el prompt al pie de la letra; diálogo fuera del inglés
Runway Gen-4.5	12 créditos por segundo	Control de cámara, referencia de personaje	Los créditos se gastan rápido

SORA YA NO EXISTE

Cierre anunciado el 24-mar-2026; API desconectada el 24-sep-2026.

CONSEJO

Separa guion, prompts, montaje y archivos fuente para poder cambiar de proveedor.

Precios oficiales de la API de Google (por segundo con audio, a 17-oct-2026)

OPCIÓN	PRECIO	CLIP DE 8 S
Veo 3.1 Standard	0,40 $/s en 720p y 1080p; 0,60 $/s en 4K	3,20 $
Veo 3.1 Fast	0,10 $/s en 720p; 0,12 $/s en 1080p; 0,30 $/s en 4K	0,80 $
Veo 3.1 Lite	0,05 $/s en 720p; 0,08 $/s en 1080p	0,40 $
Gemini Omni Flash	~0,10 $/s en 720p	~0,80 $
Runway Gen-4.5	12 créditos/s	96 créditos

Runway: Free 125 créditos una vez; Standard 12 $/mes (pago anual); Pro 28 $/mes; Max 76 $/mes. La capa gratuita de Google no incluye Veo.

PRESUPUESTA POR INTENTOS, NO POR CLIPS FINALES

Una escena con dos personajes hablando puede pedir 8-10 intentos. Explora con el modelo barato y reserva el caro para la versión final.

Cómo escribir un prompt de vídeo

1. **Vocabulario de cine.** Travelling lateral lento, zoom lento hacia el rostro, plano cenital que desciende, cámara en mano con ligero temblor.
2. **Tiempo y estabilidad.** Indica cuándo empieza el movimiento y qué no debe cambiar.
3. **Diálogo.** Texto exacto entre comillas y el tono; las frases cortas mejoran la sincronía labial.
4. **Coherencia entre clips.** Imagen de referencia fija en cada generación; para vídeos largos, el último fotograma de un clip como inicio del siguiente.

1.3 Herramientas gratis complementarias

HERRAMIENTA	PARA QUÉ	LICENCIA Y NOTAS
HyperFrames	HTML a MP4	Apache 2.0

HERRAMIENTA	PARA QUÉ	LICENCIA Y NOTAS
Lemo-Opuscar	51 estilos de cine (óleo, acuarela, tinta china, ukiyo-e, pizarra). Canvas y WebGL, música y voz sintetizadas	MIT. Node 20+ y Python 3.11+. `claude plugin marketplace add lemomo-ai/lemo-opuscar`
onetake	Plano secuencia de 10 a 30 s sin cortes	PolyForm Noncommercial: gratis solo para uso personal; para trabajo comercial hay versión Pro de pago
Claude Animation Base	Animar a Clawd (mascota de Claude) con aspecto pintado a mano, p5.js	MIT. Node, Chrome y ffmpeg
Collage B-roll	Recortes de papel con semitono (~5 s)	Animación con Gemini: clave de Google AI Studio, Python 3.10+, ffmpeg
Claude Motion	Animación explicativa desde una idea, editable	Beta, solo planes Team y Enterprise
SupoClip	Vídeo largo a clips verticales 9:16 con subtítulos	AGPL-3.0. Auto-alojado: Docker + clave AssemblyAI + clave de IA u Ollama
Shotcut	Editor de vídeo	GPL-3.0. Vertical: proyecto 1080x1920
OBS Studio	Grabar pantalla o emitir	GPL-2.0
HandBrake	Comprimir y convertir	Preset Fast 1080p30
VoiceStudio	Clonar voz, doblar y transcribir en local	AGPL-3.0. Solo con tu voz o con permiso
YouTube Agent Skill	11 skills: guion con 21 fórmulas de gancho, título y miniatura, retención, cortes, Shorts	MIT. `/plugin marketplace add Jakeschincariol/youtube-agent-skill`

Derechos y etiquetado (UE)

OBLIGACIONES AL PUBLICAR

- **Artículo 50 del Reglamento europeo de IA** (desde el 2-ago-2026): quien publica un deepfake debe avisar a la audiencia de que es artificial; la marca técnica (SynthID) no sustituye ese aviso.
- No generar a personas reales diciendo lo que no dijeron ni clonar voces sin consentimiento.
- Revisar la licencia de cada modelo antes de monetizar.

PARTE 2

Power Automate con el servidor MCP de flujos

02

Describes el flujo en lenguaje natural y el agente lo **construye, edita, ejecuta, depura y documenta**. Instalación, entornos y cinco demos reales.

2.1 Qué es

Un plugin de Microsoft (marketplace oficial de Power Platform Skills) que conecta un agente de código, como Claude Code o GitHub Copilot CLI, con Power Automate mediante el servidor MCP **Flow Agent**. Se describe el flujo en lenguaje natural y el agente lo construye, edita, ejecuta, depura y documenta. Funciona como un par de manos extra al crear flujos. Incluye 10 skills.

Qué permite

Construir
Desde una descripción: busca los conectores y acciones, comprueba las conexiones y crea el flujo.

Depurar
Analiza las ejecuciones fallidas y señala las acciones que fallaron y por qué.

Gestionar
Cancelar en bloque ejecuciones en curso o en espera, copiar flujos, listar flujos de un entorno o de todos.

Explicar y optimizar
Documenta flujos existentes, explica expresiones y crea una versión optimizada.

2.2 Instalación

1. **Añade el marketplace e instala el plugin**

```
TERMINAL

claude plugin marketplace add microsoft/power-platform-skills
claude plugin install power-automate@power-platform-skills
```

2. **Reinicia Claude Code.**

3. **Ejecuta la skill `setup` una vez.**
Inicia sesión con la cuenta de la organización y fija el entorno de trabajo.

4. **Empieza a pedir cosas en lenguaje natural.**
Hace falta una cuenta de Microsoft con acceso a Power Automate.

Skills disponibles

SETUP · BUILD-FLOW · CREATE-FLOW · DEBUG-FLOW · DIAGNOSE-FLOW · BROWSE-FLOWS · MANAGE-FLOWS · MANAGE-DESKTOP-FLOWS · ROUTE-ENVIRONMENTS · REPORT-ISSUE

Entornos

Los flujos se crean dentro de un entorno de Power Platform (`make.powerautomate.com`). El entorno de trabajo se cambia en lenguaje natural («pon mi entorno de trabajo en dev playground») y se puede dejar como predeterminado para todas las sesiones. Si se da la URL de un flujo de otro entorno, el agente cambia solo de entorno.

2.3 Demos del vídeo

Demo 1 · Formulario a SharePoint y correo

> **PETICIÓN**
> «Cuando se envíe un formulario, guarda las respuestas en SharePoint y mándame un resumen por correo».

1. **Comprueba las conexiones del entorno.** SharePoint, Outlook, OneDrive y Aprobaciones conectados; la de Microsoft Forms tenía el token caducado y se reconecta.
2. **Pregunta dónde guardar y a quién enviar.** Recibe las URL del formulario y de la lista.
3. **Descubre la estructura.** Lee las preguntas del formulario y las columnas de la lista, y mapea ambos.
4. **Valida y crea el flujo.** Estructura resultante: disparador de nueva respuesta, obtener detalles, crear elemento en SharePoint, enviar correo con el resumen.
5. **Prueba.** Se rellenó el formulario: se registró en la lista y llegó el correo.

> **IMPORTANTE**
> Todo flujo nuevo se crea **apagado** para poder revisarlo; luego se le pide que lo active.

Demo 2 · Explicar y optimizar un flujo existente

Se pasa la URL de un flujo antiguo (*work progress tracker info*) y se pide explicarlo. Exporta cada lunes una lista de SharePoint a un Excel en OneDrive y lo envía por correo. El agente detecta cuatro cuellos de botella:

- Un bucle `Apply to each` que añade filas una a una a una tabla de Excel (muchas llamadas de API).
- Una espera fija de 30 segundos como apaño para la latencia de guardado de Excel.
- Paginación.
- Un bucle sin concurrencia.

Propone y construye una versión optimizada que sustituye el bucle y la espera por una acción `Create CSV table` y una sola escritura de archivo.

FILAS EXPORTADAS	TIEMPO	LLAMADAS
+10.000	21 s	1 escritura

También ofrece alternativas (mantener CSV con el flujo encendido o apagado, o una versión Excel con Office Scripts) y pregunta qué hacer con el flujo anterior.

Demo 3 · Diagnosticar fallos

Preguntar «por qué falla este flujo»: el agente analiza el historial de ejecuciones, encuentra la causa y avisa de las conexiones rotas que hay que arreglar, en vez de revisar cada fallo a mano.

Demo 4 · Caso avanzado: importación paralela por lotes

Flujo «parallel batch import»: importa un CSV muy grande a una lista de SharePoint con la API de lotes (100 filas por llamada). El agente lo analizó, lo probó con archivos de ejemplo y propuso mejoras: validar la respuesta de la API de lotes y añadir manejo de errores.

Se añadió lógica que lee aciertos y fallos y escribe un archivo en una carpeta de registros correctos y otro en una de registros fallidos, con el motivo (por ejemplo, texto en una columna de fecha o de número).

> **RESULTADO**
> Un archivo de 10.000 filas tardó unos 5 minutos.

Demo 5 · Documentación y vista de entornos

Documentó un flujo hecho con el nuevo diseñador de workflows de Copilot Studio (entradas, representación visual y detalle de cada acción). También lista los flujos del entorno ordenados por ejecuciones, busca qué flujos fallan en todos los entornos y resume las ejecuciones de los últimos 28 días.

2.4 Cómo usar todo esto en un portfolio

Proyectos demostrables con los dos temas (Partes 1 y 2). Se pueden subir como casos de estudio con captura, vídeo y explicación breve de la decisión técnica.

	PROYECTO	QUÉ SE ENSEÑA
1	**Vídeo «Claude en el Antiguo Egipto»**	51 s con voz en español (edge-tts, `es-ES-AlvaroNeural`), subtítulos sincronizados, sprites de píxeles y música generada con ffmpeg. Hecho con HyperFrames.
2	**Vídeo «Claude y el chico en su habitación»**	Habitación 3D con CSS (`perspective` y `translate3d`) y sprites, 32 s con música.
3	**Vídeo «Trata bien a tu IA»**	Sketch 2D con sprites, 25 s.
4	**Flujo formulario a SharePoint y correo**	Creado con el agente: capturas del flujo y del resultado.
5	**Flujo optimizado de exportación CSV**	Antes (bucle y espera) frente a después (21 s para 10.000 filas).
6	**Importación por lotes con registro**	Diagrama del flujo y archivos de resultado de aciertos y fallos.

Estructura sugerida del portfolio

1. **Portada breve.** Quién eres y qué haces (vídeo con IA y automatización).
2. **Un caso por proyecto.** Problema, herramienta usada, proceso, resultado y enlace o vídeo.
3. **Sección de herramientas.** HyperFrames, Claude Code, edge-tts, ffmpeg, Power Automate MCP.
4. **Contacto.**

Pistas para pedírselo a Claude

PROMPT

«Con este PDF como contexto, diseña un portfolio de una página con estos seis casos y escribe el texto de cada uno.»

PROMPT

«Convierte cada caso en una tarjeta con título, herramientas, resultado y un espacio para el vídeo.»

PROMPT

«Escribe la sección Sobre mí centrada en vídeo con IA y automatización de procesos.»

NOTA

Los vídeos generados con voz sintética o personajes ficticios no requieren aviso por deepfake, pero conviene indicar en el portfolio que están hechos con IA.

PARTE 3 · CASO DE PORTFOLIO

La Fábrica de agentes

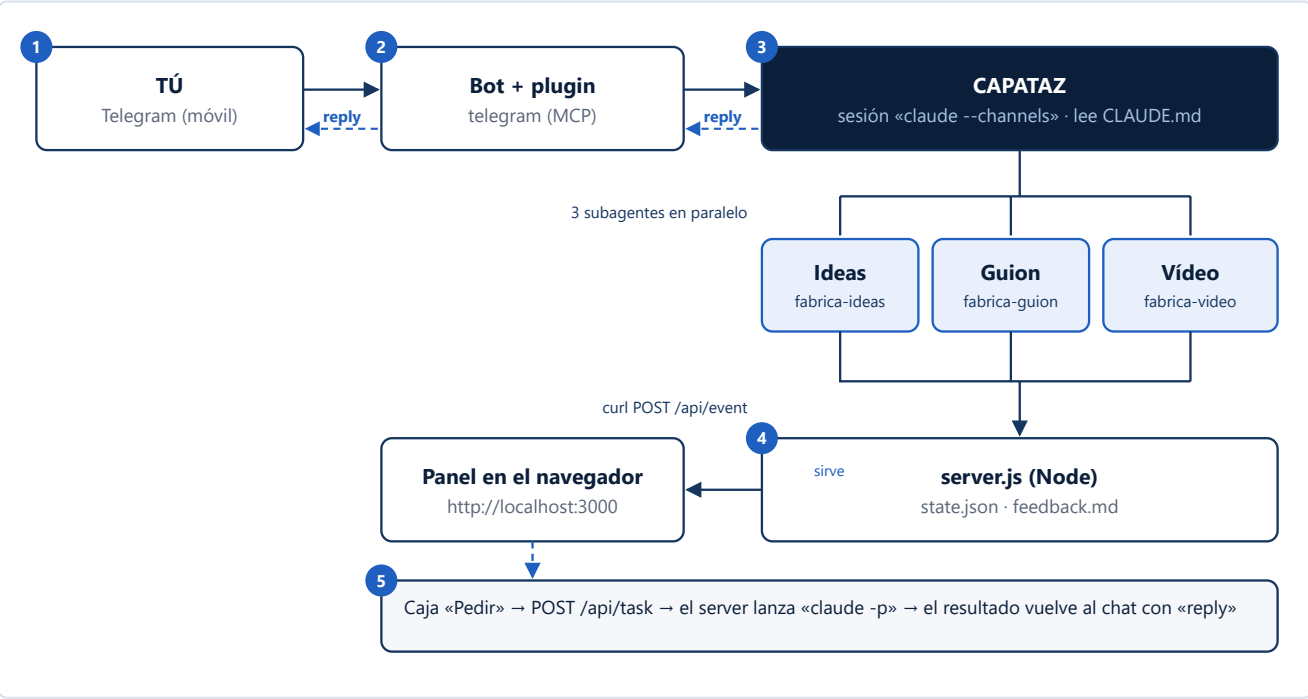
03

Una «fábrica» de contenido controlada desde **Telegram** por varios agentes de IA, con un panel visual que muestra en vivo qué línea trabaja y en qué paso. Cómo está construida y cómo reproducirla desde cero, en orden y con los comandos exactos.

3.1 Ficha del caso (para la tarjeta del portfolio)

TÍTULO	**La Fábrica de agentes: Shorts y Reels desde el móvil**
HERRAMIENTAS	Claude Code, plugin de Telegram, skills propias, Node.js (sin dependencias), HTML/CSS, Impeccable (diseño)
RESULTADO	Escribes «ideas» al bot de Telegram; tres agentes (Ideas, Guion, Vídeo) trabajan en paralelo, el panel web muestra el avance con una línea de montaje animada y el resultado llega de vuelta al chat. Tus comentarios se guardan y mejoran la siguiente tanda.
ESPACIO DE VÍDEO	Grabación de pantalla: mensaje en Telegram, panel en marcha y entrega final.

3.2 Cómo encaja todo



1 **Tú (Telegram)** escribes al bot; el plugin `telegram` (MCP) entrega el mensaje a la sesión `claude --channels`.

2. **La sesión es el capataz** (lee `CLAUDE.md`) y lanza 3 subagentes en paralelo: Ideas, Guion y Vídeo, cada uno con su skill `fabrica-*`.
3. **Cada agente avisa de su paso** con `curl POST /api/event`.
4. `server.js` **guarda** `state.json` y `feedback.md` y sirve `http://localhost:3000`.
5. **En el navegador**, la caja «Pedir» hace `POST /api/task` y el server lanza `claude -p`.
6. **El resultado vuelve a tu chat** con la tool `reply` de Telegram.

> **IDEA CLAVE**
> El cerebro es siempre Claude. El panel es solo el mando y la pantalla; el bot es solo el canal. Cada fábrica es una receta (skill): entra algo, pasa por pasos, sale algo, tú lo revisas y tus comentarios la mejoran.

3.3 Requisitos

- **Claude Code** instalado (`npm i -g @anthropic-ai/claude-code`) y sesión iniciada.
- **Bun** (lo usa el plugin de Telegram): `npm i -g bun`. **Node.js 20** o superior para el panel.
- Cuenta de **Telegram** para crear el bot. Windows, Mac o Linux.
- Las skills `fabrica-ideas`, `fabrica-guion` y `fabrica-video` (ver Paso 4).

PASO 1 Crear el bot de Telegram

1. **Abre @BotFather** en Telegram y envía `/newbot`. Elige nombre y usuario (acaba en «bot»).
2. **Guarda el token** que te da BotFather, con forma `123456789:AA...`. Es una contraseña: no la pegues en chats ni repositorios.
3. **Si se filtra**, usa `/revoke` en BotFather y genera uno nuevo.
4. **Comprueba que el token vale.** Debe devolver `"ok":true`:

```
TERMINAL

curl https://api.telegram.org/bot<TOKEN>/getMe
```

PASO 2 Conectar Telegram con Claude Code

Instala el plugin oficial y guarda el token donde el servidor lo lee al arrancar.

1. **Instala el plugin y guarda el token en la carpeta del canal**

```
TERMINAL

claude plugin install telegram@claude-plugins-official
mkdir -p ~/.claude/channels/telegram
echo "TELEGRAM_BOT_TOKEN=123456789:AA..." > ~/.claude/channels/telegram/.env
```

2 **Define quién puede hablar con el bot (allowlist).** Manda cualquier mensaje al bot y mira tu ID numérico:

```
TERMINAL
curl https://api.telegram.org/bot<TOKEN>/getUpdates    # busca "from":{"id":NUMERO}
```

3 **Escribe el archivo de acceso**

```
~/.CLAUDE/CHANNELS/TELEGRAM/ACCESS.JSON
{ "dmPolicy": "allowlist", "allowFrom": ["TU_ID_NUMERICO"], "groups": {} }
```

> **ALTERNATIVA SIN EDITAR JSON**
> Manda un DM al bot, copia el código de emparejamiento que responde y ejecútalo en Claude con `/telegram:access pair CODIGO`.

4 **Arranca el canal** en una terminal normal, no dentro de otra sesión:

```
TERMINAL
claude --channels plugin:telegram@claude-plugins-official
```

Errores que aparecieron y su causa

SÍNTOMA	CAUSA Y ARREGLO
Servidor telegram «Connection closed»	El servidor sale si no hay token al arrancar. Crea el `.env` **antes** de lanzar `claude` y reinicia la sesión.
Claude contesta en la terminal pero no llega a Telegram	El texto normal no viaja al chat: hay que usar la tool `reply` del plugin. Se fija con una regla en `CLAUDE.md` (Paso 5) y permitiendo la tool en settings.
El bot no reacciona	Tu ID no está en `allowFrom`, o hay otro proceso usando el mismo token (Telegram solo admite un consumidor por bot). Cierra sesiones viejas.
Pide permiso cada vez	Añade a `permissions.allow` de `settings.json`: `mcp__plugin_telegram_telegram__reply`, `__react`, `__edit_message`, `__download_attachment`.

PASO 3 La carpeta de la fábrica

```
TERMINAL
mkdir ~/fabrica ; mkdir ~/fabrica/app
```

```
ESTRUCTURA
fabrica/
  CLAUDE.md        <- reglas del capataz (Paso 5)
  feedback.md      <- comentarios del dueño; los agentes lo leen antes de cada tanda
  app/
    server.js      <- panel + API (Paso 6)
    index.html     <- interfaz (Paso 7)
    state.json     <- se crea solo: estado de cada fábrica y registro
```

IMPORTANTE

Lanza **siempre** `claude` desde la carpeta `fabrica`: así carga su `CLAUDE.md`.

PASO 4 Las fábricas (skills)

Una fábrica es una skill: una carpeta con un `SKILL.md` que describe entrada, pasos, rúbrica y cómo aprende. Tres de ejemplo:

FÁBRICA	ENTRA	PASOS	SALE
Ideas	Tema o nicho	Materiales, 20 ideas, filtrar con rúbrica, simular público, top 5	Top 5 numerado
Guion	Idea ganadora	Escenas y diálogos, subtítulos y sonidos, puntuar, entrega	Guion por escenas
Vídeo	Guion final	Imágenes, voz, montar con HyperFrames, revisión	MP4 vertical 9:16

REGLA DE ORO DE LA FÁBRICA: MEDIR, APRENDER Y MEJORAR

Cada skill termina pidiendo nota (1 a 10) y guardando los comentarios en `feedback.md`; la siguiente ejecución los lee primero.

Para crear una skill nueva: carpeta `~/.claude/skills/fabrica-X/SKILL.md` con cabecera `name` y `description`, o pídele a Claude: «crea una skill fabrica-X con estos pasos y esta rúbrica».

PASO 5 El capataz (CLAUDE.md)

Archivo `fabrica/CLAUDE.md`. Es lo que convierte una sesión normal en un equipo de agentes:

FABRICA/CLAUDE.MD

```
# Capataz de la fábrica (Telegram)
Eres el capataz. El dueño habla contigo SOLO por Telegram (<channel source="telegram" ...>).
Regla 1: TODA respuesta va con la tool `reply` de telegram (chat_id del mensaje).
Regla 2: al recibir un mensaje, pon `react` y avisa en una línea qué fábrica arrancas.

## Fábricas (skills)
- "ideas"   -> Skill fabrica-ideas  -> envía el top 5 numerado
- "la 3"    -> Skill fabrica-guion  -> envía guion por escenas
- "monta"   -> Skill fabrica-video  -> envía el mp4 (reply con files)
- "estado"  -> resume en qué paso está cada fábrica

## Agentes
No haces el trabajo tú: lanzas un subagente (tool Agent) por fábrica: Ideas, Guion, Vídeo.
Pueden ir en paralelo. Cada uno devuelve su resultado y tú lo envías prefijado con su nombre.

## Panel (http://localhost:3000)
Cada agente informa al INICIO de cada paso (5 pasos por fábrica):
curl -s -XPOST localhost:3000/api/event -d
  '{"factory":"ideas","status":"working","step":2,"input":"tema","note":"qué hace"}'
status: working | review | done | error. Al acabar: status review, step 5 y output con el resultado.
Antes de cada tanda lee feedback.md y aplícalo.
```

PASO 6 El servidor del panel (server.js)

Sin dependencias: solo Node. Guarda el estado en JSON, expone 4 rutas y sirve la página.

RUTA	QUIÉN LA USA Y PARA QUÉ
`GET /api/state`	La interfaz: devuelve el estado de todas las fábricas.
`POST /api/event`	Los agentes: informan de su estado y paso.
`POST /api/comment`	Comentario de revisión, se añade a `feedback.md` .
`POST /api/task`	Pedir trabajo desde el panel (lanza `claude -p`).

Código completo tal como se usa:

APP/SERVER.JS

```
// Panel de fábricas. Sin dependencias: node server.js  ->  http://localhost:3000
const http = require('http'), fs = require('fs'), path = require('path');
const PORT = process.env.PORT || 3000;
const STATE_FILE = path.join(__dirname, 'state.json');
const FEEDBACK = path.join(__dirname, '..', 'feedback.md');

const FACTORIES = {
  ideas: { name: 'Fábrica de Ideas', agent: 'Ideas',
    steps: ['Materiales', 'Generar 20 ideas', 'Filtrar con rúbrica', 'Simular público', 'Top 5'] },
  guion: { name: 'Fábrica de Guiones', agent: 'Guion',
    steps: ['Idea ganadora', 'Escenas y diálogos', 'Subtítulos y sonidos', 'Puntuar', 'Entrega'] },
  video: { name: 'Sala de Montaje', agent: 'Vídeo',
    steps: ['Recibe guion', 'Imágenes', 'Voz', 'Montar', 'Revisión'] },
};

let state = { factories: {}, log: [] };
try { state = JSON.parse(fs.readFileSync(STATE_FILE, 'utf8')); } catch {}
for (const [id, f] of Object.entries(FACTORIES))
  state.factories[id] = { status: 'idle', step: 0, input: '', output: '', comments: [],
    ...state.factories[id], ...f };

const save = () => fs.writeFileSync(STATE_FILE, JSON.stringify(state, null, 2));
const log = (factory, msg) => {
  state.log.unshift({ t: new Date().toISOString(), factory, msg });
  state.log.length = Math.min(state.log.length, 100);
};
const body = req => new Promise(r => {
  let b = ''; req.on('data', c => b += c);
  req.on('end', () => { try { r(JSON.parse(b || '{}')); } catch { r({}); } });
});
const send = (res, code, obj) => {
  res.writeHead(code, { 'Content-Type': 'application/json' }); res.end(JSON.stringify(obj));
};

http.createServer(async (req, res) => {
  if (req.method === 'GET' && req.url === '/api/state') return send(res, 200, state);

  if (req.method === 'POST' && req.url === '/api/event') {   // lo llaman los agentes
    const e = await body(req), f = state.factories[e.factory];   //
{factory,status,step,input,output,note}
```

```
    if (!f) return send(res, 400, { error: 'factory?' });
    for (const k of ['status', 'step', 'input', 'output']) if (e[k] !== undefined) f[k] = e[k];
    log(e.factory, e.note || `${f.status} · paso ${f.step}/${f.steps.length}`);
    save(); return send(res, 200, { ok: true });
  }

  if (req.method === 'POST' && req.url === '/api/task') {    // pedir trabajo desde el panel
    const t = await body(req), f = state.factories[t.factory];
    if (!f || !t.text) return send(res, 400, { error: 'factory/text?' });
    f.status = 'working'; f.step = 1; f.input = t.text; f.output = '';
    log(t.factory, `pedido: ${t.text}`); save();
    const p = require('child_process').spawn('claude', ['-p'], {
      cwd: path.join(__dirname, '..'), shell: true,
      stdio: ['pipe', 'ignore', 'ignore'], detached: true });
    p.stdin.end(`Eres el agente de la fábrica "${t.factory}". Pedido del dueño: ${t.text}\n` +
      `Usa la skill fabrica-${t.factory}, sigue CLAUDE.md (informa al panel con curl en cada paso) ` +
      `y al terminar deja status "review" con el resultado en output.`);
    p.unref(); return send(res, 200, { ok: true });
  }

  if (req.method === 'POST' && req.url === '/api/comment') { // comentario de revisión -> feedback.md
    const c = await body(req), f = state.factories[c.factory];
    if (!f || !c.text) return send(res, 400, { error: 'factory/text?' });
    f.comments.unshift({ t: new Date().toISOString(), text: c.text });
    fs.appendFileSync(FEEDBACK, `\n- [${c.factory}] ${new Date().toISOString().slice(0, 10)}:
${c.text}`);
    log(c.factory, `comentario: ${c.text}`); save(); return send(res, 200, { ok: true });
  }

  fs.readFile(path.join(__dirname, 'index.html'), (err, d) => {
    res.writeHead(err ? 404 : 200, { 'Content-Type': 'text/html; charset=utf-8' }); res.end(d);
  });
}).listen(PORT, () => console.log('Fábrica en http://localhost:' + PORT));
```

PASO 7 La interfaz (index.html)

Un solo archivo HTML con su CSS y JS. Consulta `/api/state` cada 2 segundos y redibuja solo si algo cambió (así no se pierde lo que estás escribiendo). Estructura:

Cabecera
Torre de luz (rojo, ámbar, verde) y contadores: trabajando, por revisar, entregadas, fallos.

Una fila por fábrica
Orden de trabajo a la izquierda, cinta con 5 estaciones en el centro, bahía de entrada y salida con comentario de revisión a la derecha.

Estados con color
Ámbar trabaja, azul revisa, verde entregado, rojo falla, gris espera. La estación activa se enciende; la cinta se mueve y pasan cajas mientras trabaja.

Registro de actividad
Al final de la página. Respeta «reducir movimiento» del sistema y se apila en móvil.

Dirección de diseño elegida: tablero *andon* de planta (los paneles luminosos de las fábricas reales). Fondo grafito, letra industrial condensada (Barlow Condensed), iconos dibujados en SVG, sin tarjetas iguales en cuadrícula. Se pidió con la skill Impeccable: `/impeccable QUIERO LA APP MAS VISUAL`.

PASO 8 Pedir trabajo desde el panel

La ruta `/api/task` lanza un Claude en segundo plano con el pedido, sin abrir otra terminal. El prompt va por stdin para evitar problemas de comillas en Windows:

APP/SERVER.JS (FRAGMENTO)

```
const p = require('child_process').spawn('claude', ['-p'],
  { cwd: path.join(__dirname, '..'), shell: true,
    stdio: ['pipe','ignore','ignore'], detached: true });
p.stdin.end('Eres el agente de la fábrica "ideas". Pedido: ... Sigue CLAUDE.md y deja status review.');
p.unref();
```

COSTE

Cada pedido del panel consume una ejecución de Claude (cuenta en tu uso).

3.4 Uso diario

1. **Terminal 1: el panel.** Abre `http://localhost:3000`.

TERMINAL 1

```
cd ~/fabrica/app && node server.js
```

2. **Terminal 2: el capataz con Telegram.**

TERMINAL 2

```
cd ~/fabrica && claude --channels plugin:telegram@claude-plugins-official
```

3. **Desde el móvil:** «ideas sobre ahorro» → «la 3» → «monta». Mira el panel mientras trabaja y comenta lo que no te guste: la próxima vez lo aplicará.

3.5 Reconstruirlo con Claude (prompts para copiar)

Si prefieres que Claude lo genere todo, abre una sesión vacía en una carpeta nueva y usa estos prompts **en orden**:

1 · EL CAPATAZ

«Crea en ./fabrica un CLAUDE.md de capataz: solo responde por Telegram con la tool reply, lanza un subagente por fábrica (ideas, guion, video) y cada agente informa al panel con curl POST a localhost:3000/api/event en cada paso.»

2 · EL SERVIDOR

«Crea ./fabrica/app/server.js en Node sin dependencias con GET /api/state, POST /api/event, POST /api/comment (añade a ../feedback.md) y POST /api/task (lanza claude -p con stdin). Estado en state.json y registro de 100 entradas.»

3 · LA INTERFAZ

«Crea app/index.html: tablero de planta con una línea de montaje por fábrica (5 estaciones, cinta animada cuando trabaja, luces ámbar/azul/verde/rojo), bahía de entrada/salida, caja Pedir y caja de comentario. Refresco cada 2 s sin perder lo que escribo.»

4 · LAS SKILLS

«Crea las skills fabrica-ideas, fabrica-guion y fabrica-video con sus pasos y una rúbrica de puntuación; al final piden nota y guardan comentarios en feedback.md.»

5 · EL DISEÑO

«Revisa el diseño con /impeccable y hazlo más visual.»

3.6 Límites y precauciones

- El capataz solo existe mientras la sesión esté abierta. Para tenerlo siempre encendido hace falta un servidor en la nube (por ejemplo Hetzner, unos 5 a 65 euros al mes según potencia).
- El token del bot da control total del bot: guárdalo solo en el `.env`, revócalo si se expone y mantén la allowlist con tu ID.
- Telegram solo permite una sesión por bot; dos sesiones con el mismo token se pisan.
- Cada tanda de agentes gasta uso de Claude; el render de vídeo consume CPU del ordenador.
- Las cifras y precios cambian: verifícalos antes de afirmarlos en el portfolio.
- En el portfolio indica que el contenido está hecho con IA.

PARTE 4

Cómo hacer mis automatizaciones 04

Guía práctica para montar y mantener automatizaciones propias. Une **n8n** (visual, gratis y auto-alojado), **Power Automate** (Parte 2) y **Claude Code** como agente que las construye.

Fuentes: guía «Instalar y ejecutar N8N GRATIS», documentación oficial de n8n y la Master Class de innovación con IA (Parte 5).

4.1 Qué herramienta usar

NECESIDAD	HERRAMIENTA	POR QUÉ
Flujos entre apps de Microsoft (Forms, SharePoint, Outlook, Excel)	Power Automate + plugin MCP (Parte 2)	Conectores nativos; Claude construye, depura y optimiza el flujo.
Flujos entre cualquier servicio, APIs, webhooks, agentes de IA	n8n	Gratis y sin límite de ejecuciones si lo alojas tú; flujos exportables en JSON.
Tareas locales con archivos, vídeo, scripts	Claude Code + skills / hooks	Corre en tu PC; las skills guardan la receta para reutilizarla.
Orquestar agentes desde el móvil	Fábrica de agentes (Parte 3)	Telegram + panel + subagentes.

4.2 Elegir dónde alojar n8n

OPCIÓN	COSTE	IDEAL PARA	NOTA
Docker Compose en tu PC	GRATIS	Aprender y probar	Necesita Docker Desktop en marcha. Se apaga con el PC.
Render (plan Hobby)	GRATIS	Primer servidor online	Se duerme tras unos 50 s sin uso; hay que mantenerlo despierto.
VPS (Hostinger, Hetzner, DigitalOcean)	DE PAGO	24/7 fiable	Opción recomendada para automatizaciones que no pueden fallar.
AWS EC2 t2.micro	GRATIS 12 MESES	24/7 un año	750 h/mes de capa gratuita; luego pasa a pago.
`npm install n8n -g`	GRATIS	Pruebas rápidas	Obsoleto desde n8n 3.0: mejor Docker Compose.

> **LICENCIAS**
> Sin licencia, n8n funciona como edición Community (gratis). Las licencias Business o Enterprise solo añaden funciones; el producto base es el mismo.

4.3 Instalación local con Docker Compose

La que usamos. Carpeta del proyecto: `C:\Users\Usuario\n8n`.

1. **Crea el archivo** `docker-compose.yml`

DOCKER-COMPOSE.YML

```
services:
  n8n:
    image: docker.n8n.io/n8nio/n8n
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      - GENERIC_TIMEZONE=Europe/Madrid
      - TZ=Europe/Madrid
      - N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true
      - N8N_RUNNERS_ENABLED=true
    volumes:
      - n8n_data:/home/node/.n8n
volumes:
  n8n_data:
```

2. **Abre Docker Desktop** y espera a que el engine esté en «Running».
3. **Arranca n8n** desde esa carpeta:

TERMINAL

```
docker compose up -d
```

4. **Abre** `http://localhost:5678` y crea la cuenta de propietario la primera vez.

Comandos de mantenimiento

TERMINAL

```
docker compose up -d          # arrancar en segundo plano
docker compose logs -f n8n    # ver el registro
docker compose pull ; docker compose up -d   # actualizar a la última versión
docker compose down           # parar (los datos siguen en el volumen n8n_data)
```

> **NO BORRES EL VOLUMEN**
> Los flujos y credenciales viven en el volumen `n8n_data`. `docker compose down -v` lo elimina.

Si Docker no arranca

- `docker ps` dice que no encuentra `dockerDesktopLinuxEngine`: el engine no está iniciado. Abre Docker Desktop, acepta los términos y espera a «Engine running».
- `docker desktop status` sigue en *stopped*: comprueba que WSL2 y la virtualización estén activos (`wsl --install --no-distribution` en PowerShell de administrador y reinicia).
- Para tener n8n siempre encendido aunque apagues el PC, usa un VPS (4.2).

4.4 Mantener Render despierto

1. **Crea una cuenta gratuita** en `cron-job.org`.
2. **Crea un cronjob** con la URL pública de tu n8n y ejecución cada 1 minuto.
3. **Resultado:** Render recibe una petición constante y no duerme el servidor.

4.5 Ciclo para crear una automatización

1. **Define el disparador y el resultado**
   «Cuando pase X, haz Y y avísame en Z». Una frase, sin ambigüedad.
2. **Lista los nodos antes de construir**
   Disparador, pasos intermedios, acción final, gestión de errores.
3. **Prepara las credenciales en n8n (Credentials)**
   Una por servicio. Nunca pegues claves dentro de los nodos ni en chats.
4. **Construye y prueba**
   Con «Execute workflow» y datos de ejemplo antes de activar.
5. **Activa el flujo**
   Interruptor «Active», y revisa la pestaña Executions los primeros días.
6. **Documenta y haz copia**
   Una nota por nodo (sticky note o campo «notes») y exporta el JSON como copia de seguridad.

4.6 Pedirle a Claude que cree el flujo de n8n

n8n guarda cada flujo como JSON: se puede generar con IA e importar (menú del lienzo → *Import from file*, o pegando el JSON con Ctrl+V). Esta es la instrucción de sistema de la guía original para un asistente creador de automatizaciones (por ejemplo en NotebookLM o un proyecto de Claude con plantillas):

> **INSTRUCCIÓN DE SISTEMA**
> Eres experto en crear automatizaciones JSON en N8N en JSON Blueprint. Tareas clave: traducir requisitos a JSON oficial de N8N. El resultado es SOLO JSON válido basado en plantillas y ejemplos. Documentar CADA módulo usando la estructura «notes» con explicaciones. Planifica los nodos antes de generar. Verifica la existencia de los módulos con las plantillas que tienes; básate en los [EJEMPLOS].

Plantillas para usar como ejemplos: más de 6.000 automatizaciones en la plantilla de la guía y cientos más en GitHub (enlaces en el PDF original).

> **PROMPT TIPO**
> «Con estas plantillas como ejemplo, crea un flujo de n8n que [disparador] y [acción]. Devuelve solo el JSON e incluye una nota en cada nodo.»

> **REVISA SIEMPRE EL JSON ANTES DE ACTIVARLO**
> Comprueba que los nodos existen en tu versión y que las credenciales apuntan a las tuyas.

4.7 Errores frecuentes

SÍNTOMA	CAUSA Y ARREGLO
No abre `localhost:5678`	El contenedor no corre: `docker compose ps` y `docker compose logs n8n`.
Pierdo mis flujos al reiniciar	Falta el volumen `n8n_data` o se borró con `down -v`. Haz copias exportando los JSON.
Los flujos programados no se disparan	Zona horaria mal (`GENERIC_TIMEZONE`) o el flujo no está activo.
Render deja de ejecutar	Se durmió por inactividad: configura el cronjob de 4.4.
Un nodo falla con 401/403	Credencial caducada o sin permisos: reconéctala en Credentials.
El JSON generado por IA no importa	Pide solo JSON válido, sin texto alrededor ni comas finales; usa plantillas reales de ejemplo.

PARTE 5

Prompts de innovación y marketing con IA

05

Resumen de la Master Class «Acelera el proceso de innovación con inteligencia artificial» (Dr. Juan Vicente García Manjón, 2 abril 2025). Sustituye los textos entre [corchetes] y pégalos en tu IA favorita (Claude, ChatGPT, Gemini).

Sirve para el lado de marketing de tus automatizaciones: investigar el mercado, generar ideas y diseñar el producto o la campaña.

5.1 El proceso de innovación (5 fases de IDEO)

	FASE	PREGUNTA QUE RESPONDE	PASOS
1	**Descubrimiento**	Tengo un desafío, ¿cómo lo abordo?	Comprender el desafío, preparar la investigación, recoger información
2	**Interpretación**	He aprendido algo, ¿cómo lo interpreto?	Contar historias, buscar significado, enmarcar oportunidades
3	**Ideación**	Veo una oportunidad, ¿cómo la creo?	Generar ideas, refinar ideas
4	**Experimentación**	Tengo una idea, ¿cómo la construyo?	Hacer prototipos, obtener feedback
5	**Evolución**	He creado algo nuevo, ¿cómo lo evoluciono?	Evaluar aprendizajes, avanzar

5.2 De las tendencias al mercado (3 prompts en cadena)

PROMPT 1 · TENDENCIAS

Actúa como si fueses un experto en análisis de tendencias, experto en innovación en el mercado de [mercado] y en particular en los productos de [producto] y para [país o zona]. Analiza las tendencias principales que afectan a este producto y mercado en cuanto a los cambios en los gustos y hábitos de los consumidores. Haz una tabla en la que se detalle la tendencia, su descripción y una valoración de 1 a 5 según el grado de importancia.

PROMPT 2 · APLICAR UNA TENDENCIA

Teniendo en cuenta la tendencia de [tendencia elegida] haz un análisis de cómo se puede aplicar a la innovación del producto seleccionado. Haz una tabla con: 1) Nuevas necesidades emergentes que cubre el consumidor y evidencias de que están insatisfechas. 2) Cómo cambia la propuesta de valor con esta tendencia. 3) Análisis de las propuestas más innovadoras basadas en esta tendencia. 4) Ideas de conceptos de producto que apliquen esta tendencia.

PROMPT 3 · DISEÑO DEL PRODUCTO

Teniendo en cuenta la idea de concepto [idea elegida] genera un diseño del producto. Detalla el nombre, descripción, modo de empleo y envase. Genera también una imagen del producto, packaging y marca.

5.3 Problem framing (definir bien el problema)

PROMPT

Actúa como un experto en análisis de problemas y necesidades. Considerando que somos un [rol], para el problema de [problema] haz un análisis de estos puntos:

1. ¿Cuál es el problema? Descríbelo y enumera síntomas visibles.
2. ¿Por qué no lo hemos resuelto todavía?
3. ¿Cómo somos parte del problema?
4. ¿Qué suposiciones o sesgos lo rodean?
5. ¿Quién lo experimenta y cómo lo vive? ¿Cuándo y dónde se manifiesta? ¿Qué consecuencias tiene?
6. ¿Quién más tiene este problema y cómo lo afronta?
7. ¿Quién no lo tiene y por qué? ¿Quién ha sido excluido del análisis? ¿Quién se beneficia de que exista y quién de que desaparezca?

Ahora reencuadra («reframe»): reformula el problema en una frase clara y hazlo accionable con una pregunta tipo «¿Cómo podríamos…?».

5.4 Creatividad: generación de ideas

ANALOGÍAS

Actúa como un experto en creatividad, en particular en el uso de analogías como fuente de ideas. Para el reto de [reto], genera analogías basadas en estos ámbitos: [naturaleza, colores, lenguaje…]. Para cada ámbito busca las conexiones con el problema. Para cada analogía explica: 1. Qué elemento o sistema has elegido. 2. Qué principios clave tiene y cómo funciona. 3. Cómo se aplica al problema. Genera al menos cinco analogías y propón soluciones basadas en ellas.

SEIS SOMBREROS PARA PENSAR

Analiza el reto de [reto] con la técnica de los Seis Sombreros. **Azul:** define el objetivo y estructura la sesión. **Blanco:** hechos y datos, qué sabemos y qué falta. **Rojo:** emociones e intuiciones. **Negro:** riesgos y posibles errores. **Amarillo:** beneficios y oportunidades. **Verde:** ideas nuevas y alternativas. Cierra con el Azul: resume lo aprendido y decide los próximos pasos.

5.5 Diseño de producto

PROMPT

Actúa como un experto en diseño de producto. Considerando la idea de [idea] dirigido a [usuario o cliente]:

1. Identifica la necesidad o problema concreto y quién lo experimenta.
2. Explica la función principal y qué tareas permite hacer mejor.
3. Describe forma, tamaño, materiales, tecnología y acabado, destacando lo diferencial.
4. Define qué aporta emocionalmente y qué valores representa.

5. Formula la propuesta de valor: «Nuestro producto ayuda a [usuario] a [necesidad] mediante [solución]».
6. Explica el contexto de uso y qué dice el producto de la persona que lo usa.
7. Indica en qué se diferencia de otras soluciones.
8. Identifica barreras de adopción (precio, curva de aprendizaje, compatibilidad, confianza) y cómo superarlas.
9. Resume el concepto en una frase memorable, como el titular de un pitch.

5.6 Cómo automatizar estos prompts

1. **En n8n.** Nodo Manual Trigger o Webhook (recibe mercado y producto) → nodo AI Agent / Basic LLM Chain con el Prompt 1 → segundo nodo con el Prompt 2 usando la salida del primero → guardar en Google Sheets o Notion y avisar por Telegram o correo.
2. **En Claude Code.** Guarda la cadena como skill (por ejemplo `fabrica-ideas`) para lanzarla desde Telegram o el panel de la Parte 3.
3. **Guarda siempre el resultado y tu nota** en un archivo (`feedback.md`) para que la siguiente ejecución mejore.

AVISO

Las cifras, precios y versiones cambian a menudo; verifícalos antes de usarlos. El contenido generado con IA debe indicarse como tal.

PARTE 6 · CASO DE PORTFOLIO

FitBattle Gym

06

Proyecto universitario (Sistemas, caso extra) de **Alain Garzón Perez**: base de datos de una app móvil de entrenamiento gamificada. Muestra diseño de datos (modelo entidad-relación y 3FN) y una idea de producto completa.

Fuente: FITBATTLE.pdf (6 diapositivas).

6.1 Ficha del caso (para la tarjeta del portfolio)

TÍTULO	**FitBattle Gym: entrenamiento con mecánicas de videojuego**
PROBLEMA	En muchos gimnasios los datos se gestionan a mano o con herramientas desconectadas.
SOLUCIÓN	Una base de datos única y ordenada que guarda usuarios, sesiones, ejercicios, puntos, niveles y logros.
HERRAMIENTAS	Modelado entidad-relación (pata de gallo), normalización en 3FN, prototipo funcional de la app.
RESULTADO	Modelo de 7 tablas en 3FN sin relaciones M:N sin resolver, y un prototipo navegable.
ESPACIO DE VÍDEO	Grabación del prototipo y del diagrama E-R.

6.2 Cómo se identificaron las entidades

Se imaginó una entrevista con el gerente del gimnasio y cada respuesta se tradujo en entidades y atributos. Es una técnica útil para cualquier proyecto de datos: preguntar qué hay que guardar, cómo se relaciona y qué se consulta después.

TEMA	LO QUE PIDE EL GERENTE	ENTIDAD RESULTANTE
Usuario	Nombre, correo, teléfono, nacimiento, si el bono está activo y foto de perfil	`USUARIO`
Sesiones	Fecha, duración y calorías, vinculadas al usuario	`SESION ENTRENAMIENTO`
Ejercicios por sesión	Peso, repeticiones o minutos si es cardio	`SESION EJERCICIO` (tabla intermedia)
Catálogo	Lista fija de ejercicios con enlace a vídeo de la técnica	`EJERCICIO`
Gamificación	Niveles por puntos acumulados y logros al cumplir metas	`NIVEL` `LOGRO` `USUARIO LOGRO`

6.3 Entidades y atributos del modelo final

ENTIDAD	CLAVE PRIMARIA / FORÁNEAS	ATRIBUTOS (TIPO)
USUARIO	`idUsuario`	nombreCompleto (texto), email (texto/hipervínculo), telefono (texto), fechaNacimiento (fecha/hora), bonoActivo (sí/no), fotoPerfil (archivo adjunto), puntosTotales (número)
SESION ENTRENAMIENTO	`idSesion` / `idUsuario`	fechaSesion (fecha/hora), duracionMin (número), calorias (número)
EJERCICIO	`idEjercicio`	nombreEjercicio (texto), tipo (texto), musculoPrincipal (texto), videoTutorial (hipervínculo)
SESION EJERCICIO	`idSesionEjercicio` / `idSesion`, `idEjercicio`	repeticiones (número), pesoKg (número), minutos (número)
NIVEL	`idNivel`	nombreNivel (texto), puntosMinimos (número)
LOGRO	`idLogro`	nombreLogro (texto), descripcion (texto), icono (archivo adjunto)
USUARIO LOGRO	`idUsuarioLogro` / `idUsuario`, `idLogro`	fechaObtencion (fecha/hora)

Relaciones (todas 1:N, diagrama de pata de gallo)

- Un **USUARIO** tiene muchas **SESION ENTRENAMIENTO**.
- Una sesión incluye muchas **SESION EJERCICIO**; un **EJERCICIO** aparece en muchas sesiones.
- Un **NIVEL** abarca a varios usuarios.
- Un **LOGRO** lo obtienen muchos usuarios a través de **USUARIO LOGRO**.

Las relaciones M:N se resuelven con tablas intermedias.

> **DECISIÓN DE DISEÑO CLAVE**
> El nivel **no se guarda** en USUARIO. Se calcula comparando `puntosTotales` con `puntosMinimos` de NIVEL. Así no hay clave foránea redundante y se mantiene la 3FN.

6.4 Normalización (3FN)

1FN
Todos los campos son atómicos y no hay repeticiones dentro de un campo.

2FN
En las tablas con clave compuesta (SESION EJERCICIO, USUARIO LOGRO) ningún atributo depende solo de parte de la clave.

3FN
No hay dependencias transitivas; cada atributo depende únicamente de la clave primaria de su tabla.

6.5 Cómo funciona el sistema

MOMENTO	QUÉ HACE
Alta de usuario	Crea el registro en USUARIO con `puntosTotales` a cero.

MOMENTO	QUÉ HACE
Registro de sesión	Crea un registro en SESION ENTRENAMIENTO con fecha, duración y calorías.
Registro de ejercicios	Añade filas a SESION EJERCICIO con repeticiones, peso y minutos (cardio).
Catálogo	EJERCICIO da nombre, tipo, músculo principal y vídeo tutorial.
Puntos y niveles	Al terminar la sesión suma puntos a `puntosTotales` y recalcula el nivel dinámicamente.
Logros	Si se cumple una meta (nº de sesiones o puntos), registra el logro en USUARIO LOGRO con su fecha.
Historial y consultas	Sesiones, ejercicios más frecuentes, puntos y logros por usuario, para el seguimiento del entrenador.

6.6 Mejoras propuestas

- **Rutinas predefinidas:** entidad RUTINA con planes estándar que combinen varios ejercicios, y un generador de rutinas semanales personalizadas.
- **Historial médico:** campos de antecedentes o lesiones en USUARIO para adaptar los entrenamientos (dato sensible: exige consentimiento y protección reforzada según el RGPD).
- **Métricas avanzadas:** frecuencia cardíaca y calorías diarias para ajustar objetivos.
- **Informes estadísticos:** comparativas de rendimiento entre usuarios para decidir mejor en el gimnasio.

6.7 Conexión con esta guía (ideas de automatización)

1. **Con n8n (Parte 4):** un workflow que, al insertar una fila en SESION ENTRENAMIENTO, sume los puntos, recalcule el nivel y envíe un mensaje de felicitación por Telegram.
2. **Con n8n:** un aviso automático cuando se cumpla una meta y se cree una fila en USUARIO LOGRO.
3. **Con Claude Code (Parte 3 y la guía de n8n):** pedirle que genere el script SQL de las 7 tablas a partir de la sección 6.3 y los workflows anteriores a partir de la sección 6.5.

> **NOTA**
> El modelo y el prototipo son un trabajo académico; para un uso real habría que añadir seguridad, copias de seguridad y cumplimiento del RGPD.

PARTE 7 · CASO DE PORTFOLIO

07

Dashboard financiero de Inditex en Power BI

Dashboard del Grupo Inditex construido **solo con la Memoria Anual 2025** (cuentas anuales consolidadas, informe de gestión e informe de auditoría), sin datos externos. Hecho con Claude Code y el servidor MCP de modelado de Power BI.

Fuentes: Guia_Dashboard_Inditex_PowerBI.pdf y el proyecto inditex.pbip. Relacionado con las Partes 2 y 3 de esta guía.

7.1 Ficha del caso (para la tarjeta del portfolio)

TÍTULO	**Dashboard financiero del Grupo Inditex (ejercicios 2024 y 2025)**
PROBLEMA	Convertir 85 páginas de una memoria anual en un cuadro de mando que se pueda leer en minutos.
HERRAMIENTAS	Power BI Desktop (proyecto PBIP/PBIR), Claude Code, Power BI Modeling MCP, DAX, Power Query, Python (pypdf, reportlab, Pillow).
RESULTADO	Modelo de datos con más de 60 medidas DAX y un informe de 6 páginas en blanco y negro con acento rojo **#C8102E**.
ESPACIO DE VÍDEO	Recorrido por las 6 páginas del informe y por el modelo.

7.2 Principales resultados (ejercicio 2025, cierre 31/01/2026)

VENTAS
39.864 M€
+3,2 % vs 38.632 M€

EBITDA
11.267 M€
vs 10.728 M€

EBIT
7.997 M€
vs 7.554 M€

RESULTADO NETO
6.220 M€
+5,8 % vs 5.877 M€

INDICADOR	2025	2024
Ventas	39.864 M€ (+3,2 %)	38.632 M€
EBITDA	11.267 M€	10.728 M€
EBIT	7.997 M€	7.554 M€
Resultado neto	6.220 M€ (+5,8 %)	5.877 M€
Margen bruto	58,3 %	57,8 %
Ratio de liquidez	1,54x	1,61x

INDICADOR	2025	2024
Endeudamiento (pasivo / patrimonio neto)	0,75x	0,76x
ROE / ROA (saldo al cierre)	30,5 % / 17,5 %	29,9 % / 16,9 %

ALERTAS DETECTADAS

- Liquidez corriente a la baja.
- Fondo de maniobra de 5.519 M€ (frente a 6.169 M€).
- Dividendos pagados de 5.235 M€, por encima de la caja libre de 4.686 M€.

7.3 Cómo se hizo

1. **Lectura del PDF.** 85 páginas. Se extrajeron cuenta de pérdidas y ganancias, balance y flujos (págs. 16, 20, 22), ventas por zona y segmento (pág. 43), ventas por formato (pág. 76) y fotos corporativas.
2. **Validación.** Se comprobaron todos los subtotales. Hay 12 diferencias de ±1 M€, todas por redondeo del propio informe. No se modificó ninguna cifra.
3. **Modelo.** Tabla de hechos `Datos` y tabla `Ejercicio` relacionadas por ejercicio; el diseño prevé también Zonas, Formatos y Segmentos.
4. **Medidas DAX.** Cuenta de resultados, balance, liquidez y solvencia, rentabilidad, flujos de efectivo y variaciones interanuales. Todos los ratios usan `DIVIDE`.
5. **Informe.** 6 páginas con encabezados en mayúsculas, fotos, tema propio (`Inditex.json`) y selector de ejercicio.
6. **Documentación.** La guía en PDF, el script de transcripción y validación y el CSV de origen con trazabilidad a la página del PDF.

7.4 Configuración del entorno

Requisitos: Power BI Desktop (versión de la tienda de Microsoft, formato de proyecto PBIR); Claude Code con la extensión de VS Code «Power BI Modeling MCP» (`analysis-services.powerbi-modeling-mcp`); Python 3 con pypdf, reportlab y Pillow.

Servidor MCP de modelado

En el archivo de configuración de Claude Code:

CONFIGURACIÓN DE CLAUDE CODE (JSON)

```
"powerbi-modeling": {
  "type": "stdio",
  "command": "C:/Users/Usuario/.vscode/extensions/analysis-services.powerbi-modeling-mcp-1.0.0-win32-x64/server/powerbi-modeling-mcp.exe",
  "args": ["--start"]
}
```

PROBLEMA REAL QUE APARECIÓ

La ruta apuntaba a la versión 0.4.0, que ya no existía tras actualizar la extensión a 1.0.0, y el servidor daba *Connection closed*. **Tras cada actualización de la extensión hay que revisar el número de versión de esa ruta.**

Primer uso

1. **Acepta el EULA de Microsoft.** Lo debe aceptar la persona usuaria; también se puede fijar con la variable `PBI_MODELING_MCP_ACCEPT_EULA=true`.
2. **Abre Power BI Desktop con un archivo.** Sin ninguno abierto, `ListLocalInstances` devuelve 0 instancias.
3. **Lista las instancias y conecta** con `data source=localhost:<puerto>`. El puerto cambia cada vez que se abre Desktop.
4. **Si guardas el archivo con otro nombre**, la conexión queda obsoleta: vuelve a conectar.

Formato de proyecto (PBIP)

Para escribir el informe como archivos de texto se guarda como proyecto: *Archivo > Guardar como > Power BI Project (.pbip)*. Si no aparece la opción, actívala en *Archivo > Opciones > Características en vista previa* y reinicia Desktop. Estructura:

ESTRUCTURA DEL PROYECTO

```
inditex.pbip                    <- abre este archivo en Power BI Desktop
inditex.SemanticModel/          <- modelo: definition/tables/*.tmdl, relationships.tmdl
inditex.Report/                 <- informe: definition/pages/<id>/page.json
                                   y visuals/<id>/visual.json
                                   StaticResources/ (tema Inditex.json y fotos)
dashboard_inditex/              <- datos_inditex.csv, build_data.py,
                                   build_report2.py, img/
```

Flujo de trabajo seguro con Desktop abierto

Desktop mantiene el informe en memoria: si guardas desde Desktop, **sobrescribe las páginas escritas en disco**. Orden seguro:

1. **Haz los cambios de modelo con el MCP.**
2. **Pulsa Ctrl+S y cierra Desktop.**
3. **Regenera el informe:** `python dashboard_inditex/build_report2.py`.
4. **Abre** `inditex.pbip`.

> **RUTA DEL CSV**
> El modelo carga el CSV desde `C:\Users\Usuario\Downloads\dashboard_inditex\datos_inditex.csv`: no muevas esa carpeta o cambia la ruta en Power Query.

7.5 Modelo de datos y medidas DAX

TABLA	CONTENIDO	FUENTE (PDF)
Datos	Estado, clave, concepto, nota, ejercicio, importe y página del PDF. Cuenta de resultados, balance y flujos en formato largo.	Págs. 16, 20, 22
Ejercicio	2024 (cierre 31/01/2025) y 2025 (cierre 31/01/2026). Une las demás tablas.	—
Zonas	Ventas y activos no corrientes por zona geográfica.	Pág. 43
Formatos	Ventas por cadena: Zara (con Zara Home y Lefties), Pull&Bear, Massimo Dutti, Bershka, Stradivarius, Oysho.	Pág. 76

TABLA	CONTENIDO	FUENTE (PDF)
Segmentos	Ventas a terceros, BAI, ROCE, tiendas y activos por segmento operativo.	Pág. 43

DECISIÓN DE DISEÑO

Los saldos de balance (foto al cierre) y los movimientos de la cuenta de resultados conviven en la misma tabla, pero cada medida filtra por la clave de su estado, así que no se mezclan ni se duplican importes. Los subtotales del PDF se guardan como filas propias y las medidas leen el subtotal publicado, no la suma de partidas.

Grupos de medidas

GRUPO	MEDIDAS
Cuenta de resultados	Ventas, coste de la mercancía, margen bruto, gastos de explotación, EBITDA, amortizaciones, EBIT, BAI, impuesto, resultado neto
Balance	Activo y pasivo (total, corriente, no corriente), patrimonio neto, existencias, efectivo, liquidez total, deuda financiera, pasivo por arrendamiento, posición neta de caja
Liquidez y solvencia	Fondo de maniobra, ratio de liquidez, prueba ácida, ratio de tesorería, solvencia, endeudamiento, autonomía financiera
Rentabilidad	Márgenes bruto, EBITDA, EBIT y neto (%), ROA, ROE
Flujos de efectivo	Flujo de explotación, inversión y financiación, capex, dividendos pagados, caja libre, payout
Variaciones	Año anterior, variación absoluta y porcentual de ventas, EBITDA, EBIT, resultado neto, activo, patrimonio neto y flujo de explotación
Zonas, formatos y segmentos	Ventas, % del total, variación %, activos no corrientes, BAI, margen BAI, ROCE, tiendas

Patrón de las medidas

Una medida auxiliar `Año` devuelve el ejercicio en contexto y cada medida base filtra por ese año:

```dax
Año = MAX ( Ejercicio[Ejercicio] )

Ventas =
VAR a = [Año]
RETURN
    CALCULATE ( SUM ( Datos[Importe] ), Datos[Clave] = "VENTAS", Ejercicio[Ejercicio] = a )
```

Si no hay selección se muestra el ejercicio más reciente y nunca se suman dos años. Una medida no puede usarse directamente dentro de un filtro booleano de `CALCULATE`; por eso el año se guarda antes en una variable. El código de arriba, con formato ampliado, es la medida real del proyecto: está en `inditex.SemanticModel/definition/tables/Datos.tmdl`, carpeta «1 Cuenta de resultados».

DEFINICIONES

- **ROA y ROE** usan el saldo al cierre, no el medio, porque el documento no incluye el balance de 31/01/2024.
- **Endeudamiento** = pasivo total / patrimonio neto e incluye arrendamientos (la deuda financiera pura es de 2 M€).
- **Caja libre** = flujo de explotación − capex − pagos por arrendamiento.

7.6 Páginas del informe

	PÁGINA	CONTENIDO
1	**Resumen ejecutivo**	Franja negra con 6 indicadores, evolución de resultados, márgenes, treemap del activo, cascada de ventas a resultado neto, foto de tienda y panel de conclusiones y alertas.
2	**Balance de situación**	Comparación de saldos de activo y de patrimonio neto y pasivo, dos treemaps y seis ratios de liquidez y estructura financiera.
3	**Cuenta de resultados**	Cascada, evolución de márgenes y tabla completa de la cuenta de pérdidas y ganancias de ambos ejercicios.
4	**Análisis financiero**	Liquidez y solvencia, rentabilidad, flujos de efectivo, y paneles de riesgos y oportunidades con cifras del PDF.
5	**Ventas por zona**	Mapa de burbujas, anillo, barras y tabla por zona geográfica.
6	**Ventas por formato**	Ventas por cadena, anillo, foto del flagship de Massimo Dutti y tabla de segmentos con BAI, ROCE y tiendas.

Estilo: blanco y negro con acento rojo **#C8102E** para alertas y bajadas, encabezados en mayúsculas, tarjetas blancas planas sobre fondo gris claro y un tema propio (`Inditex.json`). Cada página tiene un selector de ejercicio (por defecto 2025); los gráficos de evolución lo ignoran para mostrar siempre los dos años. Las fotos proceden de la propia Memoria Anual.

7.7 Limitaciones y puntos a comprobar

- **Informe no verificado visualmente:** se generó desde el esquema oficial (0 errores de validación) pero no se abrió en Desktop. Si un visual sale vacío o desplazado, se ajusta su JSON.
- **Comprueba el modelo del proyecto:** la copia revisada de `inditex.pbip` contiene en el modelo solo las tablas Datos (62 medidas) y Ejercicio con una relación; las tablas Zonas, Formatos y Segmentos descritas arriba no están en esa copia, así que las páginas 5 y 6 pueden salir vacías. Regenera con el script del flujo seguro (7.4) o recrea las tablas con el MCP.
- **Mapa:** el PDF no desglosa países; cada zona se ubica con un nombre aproximado y depende del geocodificador de Bing. El anillo y las barras no dependen de él.
- Zara, Zara Home y Lefties se publican agrupados; no hay ventas por sociedad jurídica, solo por formato y segmento.
- Los textos de conclusiones, alertas, riesgos y oportunidades están escritos a mano con cifras del PDF y no se actualizan solos.
- La cascada puede diferir ±1 M€ del resultado neto publicado, por redondeo. Con solo dos ejercicios, los gráficos de evolución tienen dos puntos y 2024 no tiene variación interanual.
- Las ventas a terceros por segmento suman 39.864 M€ tras el ajuste intersegmentos; el BAI de segmentos suma 8.019 M€ frente a 8.020 M€ publicados.

7.8 Cómo seguir

1. **Abre** `inditex.pbip`, revisa cada página y anota qué visual necesita retoque.
2. **Limpia el modelo.** Oculta o renombra tablas auxiliares y define una tabla de medidas para un modelo más limpio de cara al portfolio.
3. **Amplía el análisis** con notas de la memoria (provisiones, arrendamientos, impuestos).

4. **Publica en el servicio de Power BI** solo tras cambiar la ruta del CSV a una fuente en la nube o usar una puerta de enlace.
5. **Automatiza:** un flujo de Power Automate (Parte 2) o de n8n (Parte 4) que actualice el CSV y avise por correo cuando cambie una alerta, por ejemplo la liquidez.

Fuente de todos los datos: Memoria Anual Grupo Inditex 2025 (archivo PDF aportado). Proyecto de estudio y portfolio, no asesoramiento financiero; las cifras deben verificarse con el documento oficial.

PARTE 8

Cómo hacer los vídeos 3D (Three.js a MP4)

08

Receta para crear una animación 3D en bucle con **una sola petición a Claude**, verla en el navegador y exportarla a MP4 vertical (9:16) para Shorts, Reels o TikTok. Todo gratis: el motor es Three.js y el MP4 se saca con Chrome + ffmpeg.

Casos hechos con este método: **Ontzi** (reciclaje de envases, 10 s) y una **fábrica de coches en colores Jaguar** (30 s).

8.1 Resultado y requisitos

- **Entrega:** un `.html` que se reproduce en bucle (sin textos 3D, sin scroll) y un `.mp4` de 1080x1920 a 30 fps.
- **Para ver el HTML:** cualquier navegador con internet (carga Three.js desde un CDN).
- **Para sacar el MP4:** Node 20+, Google Chrome, ffmpeg (`winget install Gyan.FFmpeg`) y el paquete `playwright-core` (`npm i playwright-core`). Claude Code puede hacerlo todo por ti.

8.2 La plantilla de prompt

Se rellenan los cinco huecos entre corchetes. Cuanto más concreto el negocio y lo que se mueve, mejor sale.

PLANTILLA

Crea una animación 3D en bucle de **[segundos]** s de **[tu negocio]**, en un solo archivo HTML con Three.js. Vista isométrica, luz suave, colores **[tu marca]**. Que todo se mueva: **[coches, gente, máquinas]**. Cámara en órbita lenta. Sin texto en 3D. Formato vertical, sin barras de desplazamiento.

EJEMPLO RELLENADO (VÍDEO DE LA FÁBRICA)

Crea una animación 3D en bucle de **30** s de **COCHES**, en un solo archivo HTML con Three.js. Vista isométrica, luz suave, colores **JAGUAR**. Que todo se mueva: **FABRICA DE COCHES**. Cámara en órbita lenta. Sin texto en 3D. Formato vertical, sin barras de desplazamiento.

8.3 Pasos, de la idea al MP4

1. **Pega el prompt rellenado en Claude**
   Idealmente Claude Code, para que guarde el archivo. CLAUDE
2. **Abre el .html en el navegador**
   Míralo un ciclo entero. CHROME / EDGE

3 **Pide retoques en lenguaje normal**
Hasta que te guste (ver 8.6). CLAUDE

4 **Exporta a MP4**
«Expórtalo a MP4 vertical 1080x1920 a 30 fps». CLAUDE + CHROME + FFMPEG

5 **Opcional: añade música, voz o subtítulos**
Y súbelo. HYPERFRAMES / FABRICA-VIDEO

8.4 Reglas para que el bucle sea perfecto

Todo es función del tiempo
La animación se calcula como `draw(segundos)` con `u = (t mod duración) / duración`. Así el último fotograma enlaza con el primero y se puede renderizar cualquier instante.

Ciclos enteros
Cada movimiento repetitivo (brazos, engranajes, grúa) da un número entero de vueltas por bucle: `sin(u·2π·k)` con k entero.

Los saltos se esconden
Los coches o envases que «reaparecen» lo hacen dentro de naves o túneles opacos, nunca a la vista.

Cámara
Órbita lenta = una vuelta completa en todo el bucle, o un vaivén de ±20° con `sin(u·2π)`. Ambas cierran sin salto.

Vertical
Cámara ortográfica con ancho fijo (20 unidades) y el alto sale de la relación 9:16; así la escena nunca se corta por los lados.

Luz suave
`HemisphereLight` + una `DirectionalLight` con sombras. Si los colores oscuros se pierden, sube la intensidad de la luz hemisférica.

8.5 Exportar a MP4 fotograma a fotograma

Grabar la pantalla da tirones. Es mejor renderizar cada fotograma exacto: el HTML expone `window.draw(segundos)` y, si la URL termina en `#export`, no arranca el bucle por su cuenta. Un script abre la página en Chrome a 1080x1920, avanza el tiempo 1/30 s por fotograma, captura cada uno y ffmpeg los junta.

1 **Dentro del HTML (final del script):**

ANIMACION.HTML

```
window.draw = draw;   // exportar MP4 fotograma a fotograma
const loop = ms => { draw(ms / 1000); requestAnimationFrame(loop); };
if (!location.hash.includes('export')) requestAnimationFrame(loop);
```

2 **Crea** `rec.mjs` (30 fps; 300 fotogramas = 10 s, 900 = 30 s):

REC.MJS

```
import { chromium } from 'playwright-core';
import { mkdirSync } from 'fs';
mkdirSync('frames', { recursive: true });
const b = await chromium.launch({ channel: 'chrome',
  args: ['--use-gl=angle', '--enable-unsafe-swiftshader'] });
const pg = await b.newPage({ viewport: { width: 1080, height: 1920 } });
await pg.goto('file:///C:/Users/Usuario/Downloads/animacion.html#export');
await pg.waitForFunction(() => window.draw, null, { timeout: 30000 });
for (let i = 0; i < 900; i++) {
  await pg.evaluate(t => window.draw(t), i / 30);
  await pg.screenshot({ path: `frames/f${String(i).padStart(4, '0')}.png` });
}
await b.close();
```

3 **Monta el vídeo con ffmpeg:**

TERMINAL

```
ffmpeg -framerate 30 -i frames/f%04d.png -c:v libx264 -pix_fmt yuv420p -crf 18 animacion.mp4
```

> **ALTERNATIVA RÁPIDA Y SIN CÓDIGO**
> Grabar la pantalla con la Barra de juegos de Windows (Win+G) o con OBS, recortando a vertical. Más fácil, pero menos limpio.

8.6 Frases para retocar

> **PÍDESELO A CLAUDE ASÍ**
> - «Haz que la cámara dé una vuelta completa en todo el bucle.»
> - «Añade más coches / más robots / una segunda línea que vaya al revés.»
> - «Los colores se ven oscuros: aclara el suelo y usa un verde Jaguar más vivo.»
> - «La escena se corta por los lados en vertical: aleja la cámara.»
> - «El bucle da un salto en el segundo X: escóndelo dentro de una nave.»
> - «Cambia el negocio a [otro] manteniendo el estilo.»

8.7 Problemas típicos

SÍNTOMA	CAUSA Y ARREGLO
Pantalla en blanco al abrir el HTML	No hay internet para el CDN de Three.js, o el navegador bloquea módulos: abrir con Chrome/Edge y con conexión.
Salto o parpadeo al reiniciar el bucle	Algún movimiento no tiene ciclos enteros o hay un teletransporte a la vista. Pedir a Claude que lo revise.
Escena cortada en el MP4	Ancho de cámara demasiado pequeño para el formato 9:16. Aumentar el ancho fijo de la cámara ortográfica.

SÍNTOMA	CAUSA Y ARREGLO
El render tarda	900 capturas de 1080x1920 pueden llevar varios minutos; es normal. Para probar, baja a 24 fps o a 720x1280.
Pesa mucho	Subir `crf` (por ejemplo 23) en ffmpeg: menos peso, algo menos de calidad.

ARCHIVOS DE EJEMPLO

En la carpeta Descargas: `animacion-ontzi.html` / `.mp4` y `fabrica-coches-jaguar.html` / `.mp4`. El vídeo puede pasar después por la skill `fabrica-video` o por HyperFrames para añadir voz, música y subtítulos.

PARTE 9

Cómo hacer vídeos con las herramientas de El Telar

09

Resumen, con palabras sencillas, de las herramientas gratuitas de vídeo con IA que recoge la guía de El Telar (octubre de 2026). Pensado para quien nunca ha hecho un vídeo con IA: primero qué herramienta elegir, después una ficha por herramienta y al final un flujo completo y un glosario.

DATOS CAMBIANTES

Los precios, límites y funciones de estas herramientas cambian cada pocas semanas. Compruébalo siempre en la página oficial antes de depender de algo.

9.1 ¿Qué herramienta necesito?

QUIERO…	USO	TIPO
Sacar clips cortos verticales de un vídeo largo (podcast, charla, directo)	**SupoClip**	IA + web / Docker
Montar clips, audio e imágenes en una línea de tiempo	**Shotcut**	Editor gratis
Hacer un vídeo pesado más pequeño para poder subirlo	**HandBrake**	Conversor gratis
Grabar mi pantalla, mi cámara o hacer un directo	**OBS Studio**	Grabador gratis
Un vídeo de títulos, gráficos y música sin grabar nada	**HyperFrames**	Código a MP4
Un dibujo animado pintado a mano con un personaje	**Claude Animation Base**	Kit para Claude Code
Una película artística sin modelo de vídeo (51 estilos)	**Lemo-Opuscar**	Skill de Claude Code
Un teaser o demo de producto en un solo plano continuo	**onetake**	Skill de Claude
Una animación explicativa a partir de un informe o gráfico	**Claude Motion**	Función de Claude (beta)
Un collage de papel animado como plano de apoyo (B-roll)	**Collage B-roll**	Skill con Gemini
Clonar mi voz o doblar un vídeo a otro idioma	**VoiceStudio**	App de escritorio
Ayuda con guion, títulos y retención de un canal de YouTube	**YouTube Agent Skill**	11 skills de Claude

DOS IDEAS PARA NO PERDERSE

1. Las herramientas con IA suelen generar o proponer clips sueltos; alguien tiene que montarlos, y para eso sirve un editor como Shotcut.
2. Muchas de las más nuevas no usan un modelo de vídeo: Claude escribe código que dibuja y anima, y ese código se convierte en MP4. Por eso son gratis, repetibles y fáciles de cambiar.

9.2 Antes de empezar: lo que suele hacer falta

- Un **asistente de programación** como Claude Code (la mayoría de las herramientas de código lo usan). Lo que pagas, si acaso, es tu plan de Claude, no la herramienta.
- **Node.js** (versión 20 o superior; HyperFrames pide 22 o superior), **Google Chrome** y **ffmpeg** para renderizar a MP4.
- **Docker** solo si quieres instalar SupoClip en tu ordenador.

TRUCO MÁS FÁCIL

Pega en Claude el enlace del repositorio y la frase: *«Instala esta skill y confírmame que funciona»*. Él se encarga de la instalación.

9.3 SupoClip: de un vídeo largo a clips verticales

Herramienta de código abierto (`AGPL-3.0`), alternativa a OpusClip. Le das un vídeo largo y una IA (Gemini, GPT, Claude u Ollama en local) elige de 3 a 7 fragmentos a partir de la transcripción. Cada clip recibe notas de gancho, interés, valor y facilidad para compartir; se recorta en vertical 9:16 siguiendo la cara de quien habla, con subtítulos palabra a palabra y un título de gancho. Exporta con ajustes para TikTok, Reels y Shorts, sin marcas de agua.

CÓMO EMPEZAR

1. **Más rápido:** usar la web (`supoclip.com`) o la app de iPhone.
2. **En tu ordenador:** instalar Docker y seguir el README del repositorio.
3. **Claves:** necesitas una clave de AssemblyAI (transcripción) y una clave de un proveedor de IA, o Ollama en local.

LO HONESTO

Instalarlo tú es gratis y sin límite de uso, pero las claves de IA pueden costar dinero. No se han comprobado los precios de la versión alojada.

Dónde: github.com/FujiwaraChoki/supoclip

9.4 Shotcut: el editor gratis para montar lo que hace la IA

Editor de vídeo gratuito y de código abierto (`GPL-3.0`) para Windows, macOS y Linux. Permite varias pistas de vídeo y audio, cortes, transiciones, textos y filtros de color o volumen.

CÓMO EMPEZAR

1. **Descárgalo** en `shotcut.org` .
2. **Crea un proyecto** y arrastra clips, imágenes y audio a la lista de reproducción.
3. **Llévalos a la línea de tiempo** y corta con la herramienta de división.
4. **Añade transiciones** solapando clips y aplica filtros de texto, color o volumen.

5. **Pulsa Exportar**, elige un ajuste predefinido y guarda.

PARA REELS VERTICALES

Configura el proyecto en 1080 × 1920 antes de empezar.

LO HONESTO

Tiene más opciones que las apps de móvil, así que la curva de aprendizaje es mayor; un tutorial corto ayuda mucho. No hay versión para móvil.

Dónde: github.com/mltframework/shotcut · shotcut.org

9.5 HandBrake: comprime los vídeos antes de subirlos

Conversor de vídeo gratuito y de código abierto para Windows, macOS y Linux. Reduce el tamaño de los archivos y convierte entre formatos. Útil porque muchas herramientas de IA limitan el tamaño de lo que aceptas subir y los vídeos del móvil pesan mucho.

CÓMO EMPEZAR

1. **Descárgalo** en `handbrake.fr` y arrastra tu vídeo a la ventana.
2. **Elige un ajuste predefinido:** «Fast 1080p30» es un buen punto de partida; para archivos más pequeños, uno de 720p.
3. **Escoge la carpeta de destino** y pulsa Iniciar.
4. **Compara con el original;** si se nota pérdida de calidad, prueba un ajuste mejor.

También convierte formatos poco compatibles a MP4 y recorta la duración.

LO HONESTO

Cuánto se reduce depende del vídeo (en grabaciones de móvil suele ser mucho) y comprimir en exceso se nota. No convierte vídeos con protección anticopia (DRM). No hay versión para móvil.

Dónde: github.com/HandBrake/HandBrake · handbrake.fr

9.6 OBS Studio: graba y emite en directo

Programa gratuito y de código abierto (`GPL-2.0`) para grabar pantalla, cámara y micrófono, o emitir en directo. Funciona en Windows, macOS y Linux.

CÓMO EMPEZAR

1. **Descárgalo** en `obsproject.com` y deja que el asistente de configuración elija los ajustes.
2. **Crea una escena** y añade fuentes: captura de pantalla, cámara, micrófono, imagen (tu logo) o texto.
3. **Coloca la cámara** en una esquina y ajusta cada fuente arrastrando.
4. **Crea más escenas** (por ejemplo «Solo cámara» o «Pausa») para cambiar con un clic.
5. **Pulsa Iniciar grabación;** para emitir, pon tu clave de retransmisión de YouTube o Twitch en los ajustes.

PARA REELS VERTICALES

Usa 1080 × 1920 en los ajustes de vídeo.

LO HONESTO

Tiene muchísimas opciones y al principio abruma. Empieza con una escena sencilla y una prueba corta, y revisa siempre el nivel del micrófono. No hay versión para móvil.

Dónde: github.com/obsproject/obs-studio · obsproject.com

9.7 HyperFrames: vídeos escritos como una página web

Herramienta de código abierto de HeyGen (APACHE 2.0). El vídeo se escribe como una página web (HTML, CSS y animaciones) y HyperFrames la convierte en MP4, en tu ordenador. Está pensada para que la use un asistente: pides a Claude Code, Codex, Cursor o Gemini CLI un vídeo con título animado, gráfico y música, y él escribe la página y la renderiza. En la guía original, un vídeo de cuatro segundos tardó unos cinco en renderizar.

CÓMO EMPEZAR

1. **Necesitas Node.js 22** o superior.
2. **Instala el plugin en Claude Code** con los comandos de abajo.

```
TERMINAL

claude plugin marketplace add heygen-com/hyperframes
claude plugin install hyperframes@hyperframes
# o, para cualquier agente:
npx skills add heygen-com/hyperframes
```

3. **Pide:** «Hazme un vídeo de 10 s con un título animado, un gráfico y música».

LO HONESTO

Sirve muy bien para piezas cortas de redes con texto, gráficos y música. Para vídeo con personas reales hay que grabar o usar otras herramientas. No genera metraje: monta alrededor del material que aportes.

Dónde: github.com/heygen-com/hyperframes

9.8 Claude Animation Base: dibujos animados con Clawd

Kit gratuito (MIT) para que Claude anime un personaje con aspecto pintado a mano (acuarela y tinta), con p5.js y p5.brush. El protagonista es Clawd, la mascota de Claude, con hojas de modelo de sus emociones (31 en la guía de El Telar). Claude hace primero el storyboard, construye plano a plano, revisa su trabajo con hojas de contacto y guarda `out/video.mp4` .

CÓMO EMPEZAR

1. **Instala** Node.js, Google Chrome y ffmpeg.
2. **Clona el repositorio** y ábrelo en Claude Code.

```
TERMINAL

git clone https://github.com/JohnHeibel/ClaudeAnimationBase.git
cd ClaudeAnimationBase && npm install
npm run video
```

3. **Pide:** «Lee ANIMATION_GUIDE.md y haz un vídeo de 15 segundos de Clawd intentando cazar una mariposa».

4. **Para ver la demo incluida:** `npm install` y `node render.mjs --clip --out=out/video.mp4`, o abre `studio.html` en Chrome.

LO HONESTO

Sin tarjeta gráfica dedicada, los rellenos de acuarela tardan segundos por fotograma; pide al modelo que los evite en gráficas integradas. Es un proyecto técnico, pensado para usar con agentes. No es una skill que se instale: es una carpeta de proyecto.

Dónde: github.com/JohnHeibel/ClaudeAnimationBase

9.9 Lemo-Opuscar: 51 estilos de cine hechos con código

Skill gratuita (MIT) para Claude Code con 51 estilos de cine (óleo, acuarela, tinta china, ukiyo-e, pizarra…). Cada estilo trae un prompt reutilizable y un cortometraje de ejemplo. Las películas son páginas con canvas y WebGL renderizadas fotograma a fotograma, con música original a partir de sonidos libres y narración sintética; no hay modelo de vídeo. Incluye también estilos que filman material real (satélite, escáneres láser, datos abiertos).

CÓMO EMPEZAR

1. **Instala la skill** con los comandos de abajo.

TERMINAL

```
claude plugin marketplace add lemomo-ai/lemo-opuscar
claude plugin install lemo-opuscar@lemolab
```

2. **Mira la galería** y elige un estilo.
3. **Cuéntale tu historia al agente** y pídele que dirija la película con ese estilo.

LO HONESTO

Según su autor, los estilos están ajustados para Claude Opus 5.5 y otros modelos pueden no reproducirlos. La primera vez descarga herramientas (Node.js 20+ y Python 3.11+) y tarda unos minutos; renderizar fotograma a fotograma lleva su tiempo.

Dónde: github.com/lemomo-ai/lemo-opuscar

9.10 onetake: vídeos de un solo plano

Skill para Claude que hace vídeos de lanzamiento, teasers y demos (de 10 a 30 segundos) en un único plano continuo en el que cada escena nace de la anterior. Evita el efecto «pase de diapositivas»: algo de la pantalla sobrevive y se transforma en lo siguiente, y una sola cámara nunca corta. Un revisor automático mide la continuidad y rechaza los vídeos que parecen diapositivas.

CÓMO EMPEZAR

1. **Clona el repositorio** o instala la skill en tu agente.

TERMINAL

```
git clone https://github.com/feitangyuan/onetake.git ~/.claude/skills/onetake
```

2. **Mira los ejemplos** de la carpeta `cases`.

3. **Describe tu producto** y pide un vídeo de lanzamiento en un solo plano.

> **LO HONESTO**
> La licencia es **PolyForm Noncommercial**: gratis para proyectos personales, no para trabajos comerciales (para eso hay una versión Pro de pago).

Dónde: github.com/feitangyuan/onetake

9.11 Claude Motion: animaciones desde el chat de Claude

Función de Claude en `BETA`, para planes Team y Enterprise, que convierte una idea, un informe o un gráfico en una animación explicativa. No usa un modelo de vídeo: Claude escribe código que anima textos, gráficos, formas e imágenes, así que cada palabra, cifra o tiempo se puede cambiar después sin tocar el resto.

CÓMO EMPEZAR

1. **Entra en Claude con un plan Team o Enterprise.** En Enterprise, un propietario debe activarlo en Configuración de la organización > Artifacts.
2. **En el cuadro de mensaje,** elige «Output» y después «Motion».
3. **Describe** tema, duración, tono y datos que debe mostrar.
4. **Revisa y cambia** textos, cifras o tiempos.
5. **Pulsa «Export»** y elige MP4. Nacen privadas; «Share» para compartir.

> **LO HONESTO**
> No está en el plan gratuito ni en Pro y consume el límite de uso de tu plan (las animaciones largas gastan más). No genera imágenes realistas: sirve para explicar con texto y gráficos, no para personas o paisajes. Es una beta y puede cambiar: mira la ayuda oficial.

Dónde: Ayuda oficial de Claude Motion

9.12 VoiceStudio: clona tu voz y dobla vídeos

Aplicación de escritorio de código abierto (`AGPL-3.0`) que hace en tu ordenador lo que servicios como ElevenLabs hacen en la nube: clona una voz con una grabación corta, dobla un vídeo a otro idioma con la voz sincronizada, transcribe, dicta y crea audiolibros. Según su documentación admite 646 idiomas y no envía tu audio a ningún servidor.

CÓMO EMPEZAR

1. **Instálalo.** Mac/Linux: comando de instalación. Windows: el de PowerShell, o baja el instalador (.dmg / .exe, unos 270 MB) de Releases.

```
TERMINAL

# Mac / Linux
curl -fsSL https://voicestudio.sh/install | sh
# Windows (PowerShell)
irm https://voicestudio.sh/install | iex
```

2. **Al abrir «Voice cloning»** te pide instalar el modelo de voz (se descarga una sola vez).

3. **Hardware:** con Mac de Apple Silicon o tarjeta NVIDIA va rápido; sin gráfica funciona en la CPU, más despacio, y necesita unos 5 GB libres.

LO HONESTO

Clona solo tu propia voz o la de alguien que te haya dado permiso: suplantar a otra persona es un abuso y, según el caso, puede ser delito. El doblaje automático puede equivocarse, así que revísalo antes de publicar. Los modelos de voz tienen cada uno su licencia.

Dónde: github.com/debpalash/VoiceStudio · voicestudio.sh

Tres prompts para VoiceStudio (cambia lo que va entre corchetes)

1 · PREPARAR EL TEXTO PARA CLONAR TU VOZ

Necesito grabar entre 30 y 60 segundos de mi voz para clonarla con VoiceStudio. Escríbeme un texto en español natural, con frases de distinta longitud, preguntas y exclamaciones, y números y nombres propios, para que la grabación recoja bien mi entonación. Tema: [tu tema]. Tono: [cercano / profesional / divertido].

2 · TRADUCIR UN GUION PARA DOBLARLO

Traduce este guion de [idioma] a español de España para doblar un vídeo. Reglas: que suene hablado, no literal; que cada frase dure parecido a la original (para que encaje con la imagen); sin tecnicismos innecesarios; y conserva los nombres de marcas y productos. Devuélvelo frase a frase, con el original al lado.
[pega aquí el guion o la transcripción]

3 · REVISAR EL DOBLAJE ANTES DE PUBLICARLO

Te paso la transcripción original y la traducción de un vídeo doblado. Compáralas y dime: (1) frases que cambian el sentido, (2) frases demasiado largas para el tiempo que ocupan, (3) términos que suenan raros en español. Propón la versión corregida de cada una.
Original: [pega]
Traducción: [pega]

9.13 YouTube Agent Skill: 11 skills para tu canal

Paquete gratuito (MIT) de once skills para Claude que ayudan a llevar un canal de YouTube: escribe guiones a partir de 21 fórmulas de gancho y puntúa el gancho antes de grabar; revisa título y miniatura como pareja; lee la retención exportada de YouTube Studio y te dice el segundo exacto en que la gente se fue; convierte una transcripción en lista de cortes; encuentra Shorts dentro de un vídeo largo y busca qué funciona en tu nicho.

CÓMO EMPEZAR

1. **Pega en Claude Code el enlace del repositorio** y: *«Install this skill, then confirm /yt-script works.»*
2. **O instálalo como plugin:**

```
CLAUDE CODE

/plugin marketplace add Jakeschincariol/youtube-agent-skill
/plugin install youtube-agent
```

3. **Dedica diez minutos** a rellenar `templates/voice.md` con tu forma de hablar: todas las skills lo leen.

LO HONESTO

No publica nada por ti: las skills escriben y analizan, subir el vídeo lo haces tú. Varias usan herramientas en Python, así que rinde más con Claude Code; en el navegador puedes pegar el `SKILL.md` al principio del chat. Las recomendaciones son orientativas y no garantizan más visitas.

Dónde: github.com/Jakeschincariol/youtube-agent-skill

9.14 Collage B-roll: recortes de papel que se montan solos

Convierte una frase de tu guion en un plano de collage de papel con trama de semitono que se ensambla solo en unos 5 segundos. Va en tres pasos con tu aprobación: te propone la idea visual, luego la imagen fija y por último la anima.

CÓMO EMPEZAR

1. **Pega en tu IA el enlace del repositorio** y pídele que te diga qué más necesitas (claves, Python, ffmpeg).
2. **O clónalo con el comando de abajo:**

TERMINAL

```
git clone https://github.com/pyang5166/gbro-collage-broll.git ~/.claude/skills/gbro-collage-broll
```

LO HONESTO

La animación la hace Gemini (Google): necesitas una clave de Google AI Studio, Python 3.10+ y ffmpeg. Está pensada para Codex; en Claude Code la imagen fija debes resolverla con otra herramienta.

Dónde: github.com/pyang5166/gbro-collage-broll

9.15 Flujo completo de principio a fin

Una forma ordenada de combinar las herramientas para sacar un vídeo vertical publicable:

1. **Escribir el guion y el gancho**
   Primeros 3 segundos. CLAUDE YOUTUBE AGENT SKILL
2. **Grabar o generar**
   Grabar pantalla, cámara y voz, o generar el vídeo con código. OBS STUDIO HYPERFRAMES CLAUDE ANIMATION BASE ONETAKE
3. **Sacar los mejores momentos**
   Si el vídeo es largo, extraerlos en vertical. SUPOCLIP
4. **Clonar o doblar la voz**
   Si hace falta otro idioma o no quieres grabar la voz. VOICESTUDIO
5. **Montar**
   Clips, audio y subtítulos en una línea de tiempo 1080 × 1920. SHOTCUT
6. **Comprimir**
   Si pesa demasiado para subirlo a la plataforma. HANDBRAKE

7 **Revisar y publicar tú mismo**
En TikTok, Reels o Shorts.

9.16 Licencias: qué puedes hacer con cada una

LICENCIA	HERRAMIENTAS	EN CORTO
MIT / Apache 2.0	HyperFrames, Claude Animation Base, Lemo-Opuscar, YouTube Agent Skill	Uso libre, también comercial.
GPL-2.0 / GPL-3.0	OBS Studio, Shotcut	Gratis y abiertos; los vídeos que hagas son tuyos.
AGPL-3.0	SupoClip, VoiceStudio	Código abierto; si modificas y ofreces el programa como servicio, debes publicar tus cambios.
PolyForm Noncommercial	onetake	Solo uso no comercial. Para negocio, versión Pro de pago.
Servicio de Anthropic	Claude Motion	Según tu plan (Team o Enterprise).

AVISO GENERAL
Clona solo voces propias o con permiso; revisa los doblajes antes de publicar; y comprueba las normas de etiquetado de contenido generado con IA de cada plataforma. Esta parte es un resumen informativo, no asesoramiento legal.

9.17 Glosario rápido

PALABRA	SIGNIFICA
MP4	El formato de vídeo más compatible: se ve en cualquier móvil, web y red social.
9:16 / 1080 × 1920	Formato vertical de TikTok, Reels y Shorts.
Skill	Paquete de instrucciones que enseña a Claude a hacer una tarea concreta.
Repositorio / GitHub	Carpeta pública con el código de un proyecto, alojada en github.com.
Claude Code	Versión de Claude que trabaja en tu ordenador, lee y escribe archivos y ejecuta comandos.
Docker	Programa que ejecuta aplicaciones ya preparadas sin instalarlas a mano.
B-roll	Planos de apoyo que se ponen sobre la voz o el plano principal.
Renderizar	Convertir el proyecto o el código en el archivo de vídeo final.
Clave de API	Contraseña que da acceso a un servicio de IA; puede tener coste. No la compartas.

Fuente: guía de vídeo con IA de El Telar, octubre de 2026 (resumen y reorganización). Comprueba datos, licencias y precios en cada repositorio oficial.

PARTE 10

Pokémon Lumier: un fangame que se juega en el navegador

Cómo se convirtió un juego de RPG Maker (Pokémon Añil) en un juego web que funciona en PC y móvil, sin instalar nada, y cómo se publicó gratis en GitHub Pages.

10.1 Ficha del caso (para la tarjeta del portfolio)

TÍTULO	Pokémon Lumier · Ciudad Plateada
HERRAMIENTAS	Claude Code, Python (lector propio de archivos de RPG Maker), Pillow, JavaScript + Canvas 2D, esbuild, Git, GitHub Pages
RESULTADO	Juego completo de una ciudad: intro, mapas originales, NPC, tienda, mochila, Centro Pokémon, combates por turnos a nivel 100, jefes con historia y evento de Celebi. Guarda la partida en el navegador.
JUGAR	`https://guisaalain-web.github.io/alain-garzon-portfolio/demos/pokemon-lumier/`
CÓDIGO	Repositorio privado: se enseña bajo petición (guisaalain@gmail.com)

10.2 Cómo funciona por dentro



- **Los mapas son los reales.** Los archivos de RPG Maker están en formato Marshal de Ruby. Un lector propio en Python (`rmarshal.py`) los abre y dibuja cada mapa con sus tilesets y autotiles en dos capas: la de abajo y la que va por encima del personaje.
- **El mundo es un JSON.** De cada mapa salen la rejilla de paso (pared, suelo, hierba, agua), los NPC con sus textos originales y las puertas (transferencias entre mapas).
- **El motor es un solo archivo JS** (`lumier.js` , ~700 líneas): mueve al jugador casilla a casilla, dibuja mapa + personajes ordenados por altura, abre puertas, muestra diálogos y lanza los combates.
- **Los combates** usan un motor por turnos sin pantalla (`engine.js`): tipos, críticos, estados, habilidades y objetos. Emite eventos («mensaje», «daño», «debilitado») que la capa visual convierte en animaciones estilo GBA.
- **La historia son banderas** guardadas en la partida (`roldan` , `lete` , `giovanni` …). Cada evento comprueba sus banderas y se dispara una sola vez.

IDEA CLAVE

No hace falta reescribir un juego desde cero: se leen sus datos originales y se les pone un motor nuevo encima. Claude se encarga del código; tú decides la historia, los personajes y el estilo.

10.3 Cómo se hizo, paso a paso

PASO 1 Exportar los mapas del juego original

1. Lee `MapInfos.rxdata` para saber qué mapas cuelgan de la ciudad (casas, Centro, museo, gimnasio).
2. Para cada mapa: dibuja los tiles (los ≥384 vienen del tileset; los 48–383 son autotiles con la tabla estándar de 48 patrones) y separa la capa alta según su prioridad.
3. Guarda `NNN_low.png`, `NNN_high.png` y el `world.json` con rejilla, NPC, textos y puertas.

```
TERMINAL
python export_plateada.py
# 6 mapas: 9 ciudad, 36 Centro, 37 casas, 40 museo, 42 gimnasio, 178 claro
```

PASO 2 Personajes que se parecen a ti

1. Elige sprites del propio juego que se parezcan (pelo, barba, gafas) y recolorea la ropa con Pillow por tono y zona.
2. Usa la hoja del mapa (4×4 pasos) y la foto de combate del mismo personaje para que encajen con el resto.

PASO 3 Motor, eventos y combates

1. Pide a Claude el motor por partes: mapa y colisiones → diálogos → puertas → combate → tienda y mochila → guardado.
2. Cada jefe es una ficha con su equipo de 6 Pokémon a nivel 100, su música y su fondo de combate.
3. Los eventos (Misty, LETE y el Team Rocket, Giovanni, Lance y la captura de Celebi) son funciones que mueven NPC, la cámara y los diálogos.

```
TERMINAL
npx esbuild src/lumier.js --bundle --outfile=dist/lumier.js --format=iife
python -m http.server 8770      # probar en http://localhost:8770
```

PASO 4 Móvil y publicación gratis

1. Botones táctiles (cruceta, A y B) que solo aparecen en pantallas táctiles y envían las mismas teclas que el teclado.
2. Sube la carpeta a un repositorio público y activa GitHub Pages en la rama `main`.
3. Para actualizar: recompila, cambia el `?v=` del script en `index.html` (evita la caché) y haz push.

```
TERMINAL
git init -b main && git add -A && git commit -m "Pokémon Lumier"
gh repo create TU_USUARIO/pokemon-lumier --public --source . --push
gh api -X POST repos/TU_USUARIO/pokemon-lumier/pages -f "source[branch]=main" -f "source[path]=/"
```

10.4 La historia del juego (orden de eventos)

1. Intro como la de Añil (splash, créditos, aviso de fangame y título **POKÉMON LUMIER**) con menú de solo **CONTINUAR**.
2. Ciudad Plateada libre: NPC originales, objetos en el suelo, tienda con el especial **Pastel Bilbaíno**, máximo 3 de cada objeto. Solo se cura en el **Centro Pokémon**.
3. **JOKIN** («Don Prodigio», Centro Pokémon) cambia un Pokémon de tu equipo. **UNAI** (casa) reta con dragones.
4. Gimnasio: vences al líder **ROLDAN** → entra **MISTY**, avisa de gritos en el Claro y regala un **Starmie**.
5. Al salir: **LETE** y el Team Rocket. Si ganas, se chiva de Giovanni y huyen («¡Nos vemos de nuevo, frijolito!»).
6. Claro Secreto: **GIOVANNI** intenta robar a Celebi. Si ganas, **LANCE** lo detiene y tú atrapas a **Celebi** con una Poké Ball.

10.5 Errores que aparecieron y su arreglo

SÍNTOMA	CAUSA Y ARREGLO
No se podía hablar con la enfermera	Su mostrador ocupa 2 casillas y solo se miraba 1. Ahora se habla a través de mostradores de hasta 3.
Un personaje nuevo no se veía	Le faltaba el fotograma inicial de su animación (valor vacío). Se pone 0 por defecto.
Los cambios no salían en la web	El navegador usaba la versión guardada. Se añade `?v=fecha` al script en cada subida.
Seguía saliendo Pikachu o Misty curaba	Se cargaba una partida antigua. Se cambió la clave de guardado (`.save.v2`) para empezar de cero.
Mapas con letras raras al leerlos	Los textos vienen en UTF-8 dentro del formato Marshal: se decodifican uno a uno.

LO HONESTO

Es un fangame sin ánimo de lucro. Usa gráficos y música de Pokémon y de Pokémon Añil (© Nintendo / Game Freak / sus autores). Lo habitual es que se tolere, pero Nintendo puede pedir retirarlo; si pasa, GitHub desactiva la página. No se ha probado en móviles reales de todas las marcas.

PROMPT PARA EMPEZAR EL TUYO

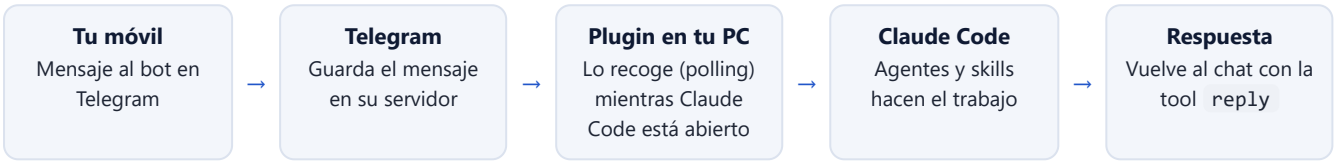
«Tengo la carpeta de un juego de RPG Maker XP en [ruta]. Lee sus mapas y eventos con un lector propio de Marshal, exporta solo [la ciudad X] a PNG + JSON y monta un juego web en Canvas 512×384 con movimiento por casillas, diálogos, puertas y combates por turnos. Protagonista [nombre], jefe [nombre].»

PARTE 11

Agentes en Telegram: cómo funcionan y por qué son una demo

La Fábrica de agentes (Parte 3) y la oficina «Alain Corporative» se manejan desde Telegram. Aquí se explica qué pasa cuando escribes al bot, por qué no contesta a cualquiera y cómo dar acceso.

11.1 Qué pasa cuando escribes al bot



- El bot es solo el **canal**. El cerebro es la sesión de Claude Code de tu ordenador.
- El plugin oficial (`telegram@claude-plugins-official`) lee el token de `~/.claude/channels/telegram/.env` y la lista de acceso de `access.json`.
- Si cierras Claude Code o apagas el PC, los mensajes se quedan en Telegram sin respuesta hasta que vuelvas a abrirlo.

11.2 Por qué en el portfolio es una demo

MOTIVO	QUÉ SIGNIFICA PARA QUIEN VISITA LA WEB
Lista de acceso (`dmPolicy: allowlist`)	Solo responde a los usuarios aprobados (ahora, 1: el dueño). A los demás no les contesta.
Depende de una sesión abierta	Responde solo cuando el ordenador y Claude Code están encendidos.
Controla un ordenador real	Cada mensaje lo atiende Claude Code con acceso a archivos y herramientas del PC. Abrirlo a cualquiera sería peligroso.

SEGURIDAD

No abras el bot a todo el mundo ni apruebes accesos porque alguien te lo pida por Telegram: es exactamente lo que haría alguien que quiere usar tu ordenador. Los accesos se gestionan siempre desde tu terminal.

11.3 Comprobar que el bot está vivo

TERMINAL

```
curl https://api.telegram.org/bot<TOKEN>/getMe            # nombre del bot y "ok":true
curl https://api.telegram.org/bot<TOKEN>/getWebhookInfo   # mensajes pendientes sin leer
```

11.4 Dar acceso a una persona concreta

1. Pide a esa persona que escriba al bot. El plugin le devuelve un código de emparejamiento.
2. En **tu terminal** (no desde Telegram) abre la gestión de accesos y aprueba ese código, o añade su usuario a la lista.
3. Prueba: que te escriba «hola» y comprueba que la respuesta llega. Para quitarle el acceso, usa el mismo comando.

```
TERMINAL
/telegram:access       # dentro de Claude Code: ver, aprobar o quitar accesos
```

11.5 Si quieres un bot público de verdad

Para que cualquiera pueda hablar con un bot desde tu web, hace falta un bot **distinto**, que no controle tu ordenador:

- **Servidor aparte** siempre encendido (n8n en Render, Parte 4, o una función en la nube) que reciba los mensajes por webhook.
- **Un modelo con instrucciones cerradas**: responde preguntas sobre tu trabajo, enseña ejemplos y deriva al email. Sin acceso a archivos ni comandos.
- **Límites**: mensajes por usuario y por día, para no gastar de más con la API.
- En la web, ese bot se anuncia como «asistente» y la Fábrica sigue marcada como **demo privada**.

11.6 Texto recomendado para la ficha del portfolio

DEMO PRIVADA

«Este bot está conectado a mi ordenador y solo responde a usuarios aprobados mientras mi sesión de Claude Code está abierta. Si quieres verlo funcionar, escríbeme a guisaalain@gmail.com y te doy acceso o te lo enseño en directo.»

11.7 Resumen

PREGUNTA	RESPUESTA CORTA
¿Funciona si un visitante escribe?	No, salvo que esté aprobado y tu sesión esté abierta.
¿Funciona para el dueño?	Sí, mientras Claude Code está abierto con el plugin de Telegram.
¿Cómo lo enseño en una entrevista?	En directo desde tu móvil, o grabando un vídeo corto del bot y del panel trabajando.

PARTE 12

Un avatar tuyo que habla con tu voz y presenta a tus agentes

Cómo se hizo el vídeo de 30 s para LinkedIn: un personaje voxel que se parece a mí, con mi voz clonada, que presenta a su equipo de agentes de IA y termina enseñando el portfolio.

12.1 Ficha del caso

FORMATO	Vídeo vertical 9:16, 30 s, con subtítulos y música
HERRAMIENTAS	Claude Code, VoiceStudio (voz clonada local), Higgsfield (GPT Image 2.5 y MiniMax H3), faster-whisper, Python + Pillow, ffmpeg
COSTE	Unos 50 créditos de Higgsfield: personaje (2 imágenes) y dos clips de 13 y 10 s. Voz, música y montaje: gratis en local.
VER	Ficha «Mi avatar con agentes (LinkedIn)» del portfolio





12.2 El recorrido

Guion + voz 70 palabras, clon local → **Personaje** Foto + estilo de referencia → **Animación** Clips con audio de referencia → **Cuadrar boca** Whisper + remapeo → **Montaje** Títulos, subtítulos, cierre

PASO 1 Guion y voz

1. Guion de unas 70 palabras para 30 s (≈2,5 palabras por segundo): quién eres, qué hace cada agente, qué ganas tú y una llamada al portfolio.
2. Genera la voz con tu clon local (VoiceStudio, perfil propio). Solo `text` + `profile_id`: con más parámetros devuelve error 400.
3. Saca los tiempos de cada palabra: sirven para los subtítulos y para el paso 4.

TERMINAL

```
from faster_whisper import WhisperModel
m = WhisperModel("small", device="cpu", compute_type="int8")
segs, _ = m.transcribe("voz.wav", language="es", word_timestamps=True)
```

PASO 2 Un personaje que se parece a ti (no tu cara pegada)

1. Sube tu foto y uno o dos fotogramas del estilo que te gusta. Pide una imagen 9:16 describiendo tus rasgos: pelo, barba, gafas, ropa.
2. Deja vacío el tercio de arriba para los títulos. Pide 2 variantes y quédate con la de la cara más grande: se anima mejor.

TERMINAL

```
3D voxel render, blocky style like the reference frames. Main character: a voxel version
of the man in the photo (short dark hair, beard, thin round gold glasses, beige shirt),
talking to camera. Around him 4 cube robot agents labeled FACTURAS, EXCEL, DOCS, INFORMES.
Keep the top third empty. Vertical 9:16.
```

PASO 3 Animarlo con tu voz

1. Parte el audio en trozos de menos de 15 s, cortando en una pausa (aquí: 13,1 s + 9,5 s).
2. Genera un clip por trozo con un modelo que acepte **audio de referencia**. En Higgsfield funcionó **MiniMax H3** (~2 créditos por segundo) con la imagen como *referencia*, no como fotograma inicial: si mezclas ambos, da error 422.
3. Pide que hable a cámara, que gesticule hacia los agentes y que cada agente brille cuando lo nombras. Mira siempre el coste antes (`get_cost`).

LO QUE NO FUNCIONÓ

Seedance 2.5 (el mejor para esto) pide un plan superior al Starter, y a 1080p costaba 368 créditos. Grok Video falló sin dar motivo (devolvió los créditos).

PASO 4 Cuadrar la boca con tu voz real

El modelo sigue el audio, pero no siempre al pie de la letra: en el segundo clip cambió el ritmo e inventó sílabas. Solución sin volver a generar:

1. Transcribe el audio del clip con faster-whisper y empareja sus palabras con las de tu voz (`difflib.SequenceMatcher`).
2. Cada pareja es un ancla (tiempo en tu voz ↔ tiempo en el clip). Para cada fotograma de salida, interpola entre anclas y coge el fotograma del clip que toca.
3. Monta con **tu** audio: la boca cae donde dices cada palabra.

TERMINAL

```
python warp.py clipA.mp4 0 13.1 wA.mp4      # clip, inicio y fin en el timeline final
python warp.py clipB.mp4 13.1 19.6 wB.mp4
```

PASO 5 Montaje y cierre

- **Títulos** en fuente pixel (Press Start 2P) con un degradado oscuro detrás para que se lean sobre la escena.
- **Subtítulos** de 3 palabras con la que suena resaltada: en LinkedIn el vídeo empieza en silencio.
- **Cierre**: captura del portfolio (Chrome headless), URL, retrato del avatar y «enlace en el post».
- **Audio**: voz + música que baja cuando hablas, normalizado a −14 LUFS.

TERMINAL

```
[m][voz]sidechaincompress=threshold=0.05:ratio=4[md];
[voz][md]amix=inputs=2:normalize=0,loudnorm=I=-14:TP=-1.5
```



12.3 Antes de publicar en LinkedIn

COMPROBACIÓN	POR QUÉ
Escucharlo entero con cascos	La voz clonada puede comerse alguna palabra.
Revisar el tramo reajustado	Donde la IA inventó sílabas, el vídeo acelera medio segundo.
Enlace en el texto del post	En el vídeo no se puede pulsar; al patrocinarlo, pon el portfolio como destino.
Indicar que está hecho con IA	Transparencia: el personaje y la animación son generados.